

**BACHELOR CONCEPTEUR DÉVELOPPEUR DE SOLUTIONS DIGITALES — TITRE 6 (RNCP
36146)**

UrbanFlow Mobility

Plateforme de mobilité urbaine intelligente

Dossier de projet — Session Septembre 2026

Yvan Kerdanet

B3 Développement — Digital Campus Paris

Sommaire

1. [Introduction générale](#)

2. [Contexte et objectifs du projet](#)

- [2.1 Présentation du commanditaire et du contexte](#)
- [2.2 Analyse de la demande client](#)
- [2.3 Cibles et personas](#)
- [2.4 Enjeux métiers et objectifs économiques](#)
- [2.5 Recueil et hiérarchisation des besoins](#)
- [2.6 Anticiper les évolutions futures](#)

3. [État de l'art et recommandations technologiques](#)

- [3.1 Benchmark concurrentiel des solutions de mobilité existantes](#)
- [3.2 Standards de données de transport](#)
- [3.3 Moteur de calcul d'itinéraires multimodaux](#)
- [3.4 Architecture applicative](#)
- [3.5 Architecture applicative : Full Stack intégré ou Frontend/Backend séparés](#)
- [3.6 Benchmark des frameworks Frontend](#)
- [3.7 Benchmark des frameworks Backend](#)
- [3.8 Benchmark des bases de données](#)
- [3.9 Benchmark des hébergeurs cloud](#)
- [3.10 Stack technique retenue — synthèse](#)
- [3.11 Synthèse des arbitrages](#)

4. [Architecture globale et spécifications fonctionnelles](#)

- [4.1 Vue d'ensemble de l'architecture](#)
- [4.2 Description des composants](#)
- [4.3 Nomenclature et conventions](#)
- [4.4 Spécifications fonctionnelles des modules principaux](#)
- [4.5 Évolutivité et maintenabilité](#)

5. [Méthodologie de gestion de projet](#)

- [5.1 Approche méthodologique retenue](#)
- [5.2 Environnement et outils de travail](#)
- [5.3 Rôles et responsabilités](#)
- [5.4 Déroulement d'un cycle d'itération](#)

6. [Démarche qualité et amélioration continue](#)

- [6.1 Indicateurs de qualité suivis](#)
- [6.2 Démarche d'amélioration continue](#)
- [6.3 Boucle de capitalisation](#)
- [6.4 Bilan quantitatif de la démarche](#)

7. [Spécifications détaillées d'une fonctionnalité clé](#)

- [7.1 Présentation et objectifs de la fonctionnalité](#)
- [7.2 Spécifications fonctionnelles](#)
- [7.3 Spécifications techniques](#)
- [7.4 Limites et évolutions possibles](#)

8. [Diagrammes UML](#)

- [8.1 Diagramme de cas d'utilisation](#)
- [8.2 Diagramme de séquence](#)
- [8.3 Diagramme de communication](#)

9. [Gestion des bogues et qualité de code](#)

- [9.1 Détection et priorisation des anomalies](#)
- [9.2 Processus de correction](#)
- [9.3 Approche spécifique à la phase de préproduction](#)
- [9.4 Cas concrets de bogues traités](#)

10. [Contraintes transverses \(sécurité, RGPD, accessibilité, éco-conception, PWA, performance\)](#)

- [10.1 Sécurité des données](#)
- [10.2 RGPD et données de géolocalisation](#)
- [10.3 Accessibilité \(WCAG 2.1 AA\)](#)
- [10.4 Éco-conception](#)
- [10.5 PWA et performance en mobilité](#)

11. [Conclusion et perspectives](#)

- [11.1 Perspectives et feuille de route post-MVP](#)
- [11.2 Écarts, frictions et enseignements \(post-mortem\)](#)

12. [Annexes](#)

1. Introduction générale

Je m'appelle Yvan Kerdanet, actuellement en troisième année de Bachelor Développeur à Digital Campus Paris, en vue d'obtenir le Titre 6 Concepteur Développeur de Solutions Digitales (RNCP 36146). Ce dossier présente le projet que j'ai mené dans ce cadre : UrbanFlow Mobility.

Le projet consiste à concevoir et développer une plateforme de mobilité urbaine intelligente pour une métropole de 500 000 habitants engagée dans sa transition écologique. L'objectif est de proposer aux citoyens un point d'entrée unique pour organiser leurs déplacements en combinant plusieurs modes de transport (vélos, trottinettes, transports en commun, covoiturage), tout en s'appuyant sur l'intelligence artificielle pour optimiser les trajets, réduire l'empreinte carbone et fluidifier le trafic urbain.

Cette étude retrace l'ensemble de la démarche suivie, depuis l'analyse du besoin client jusqu'aux choix techniques et méthodologiques retenus pour la conception et le développement de la solution.

2. Contexte et objectifs du projet

2.1 Présentation du commanditaire et du contexte

Le commanditaire du projet est une métropole de 500 000 habitants engagée dans une politique de transition écologique. Le prototype a été développé et déployé sur les données réelles de **Rennes Métropole** (flux GTFS et GBFS du réseau STAR, données OpenStreetMap), retenue comme métropole de référence : agglomération d'environ 460 000 habitants aujourd'hui, dont la trajectoire démographique la situe autour des 500 000 à l'horizon de quelques années, avec un réseau multimodal (métro, bus, vélos en libre-service) suffisamment complet pour éprouver la plateforme en conditions réalistes. Comme beaucoup de grandes agglomérations françaises et européennes, elle fait face à trois problématiques qui se renforcent mutuellement : une congestion routière chronique aux heures de pointe, un niveau de pollution atmosphérique et sonore préoccupant, et une offre de mobilité fragmentée où chaque mode de transport (bus, tram, vélos et trottinettes en libre-service, covoiturage) fonctionne avec ses propres outils, sans vision d'ensemble pour l'utilisateur.

Dans ce contexte, la métropole souhaite se doter d'une plateforme unifiée capable de repenser la façon dont ses citoyens organisent leurs déplacements, en s'appuyant sur les technologies numériques et l'intelligence artificielle. Ce projet s'inscrit dans une démarche plus large de "Mobility as a Service" (MaaS), un modèle déjà expérimenté dans plusieurs villes européennes, qui vise à agréger l'ensemble des offres de mobilité au sein d'une application centrale.

2.2 Analyse de la demande client

La demande initiale, formulée par la responsable du projet côté métropole, liste une dizaine de fonctionnalités souhaitées :

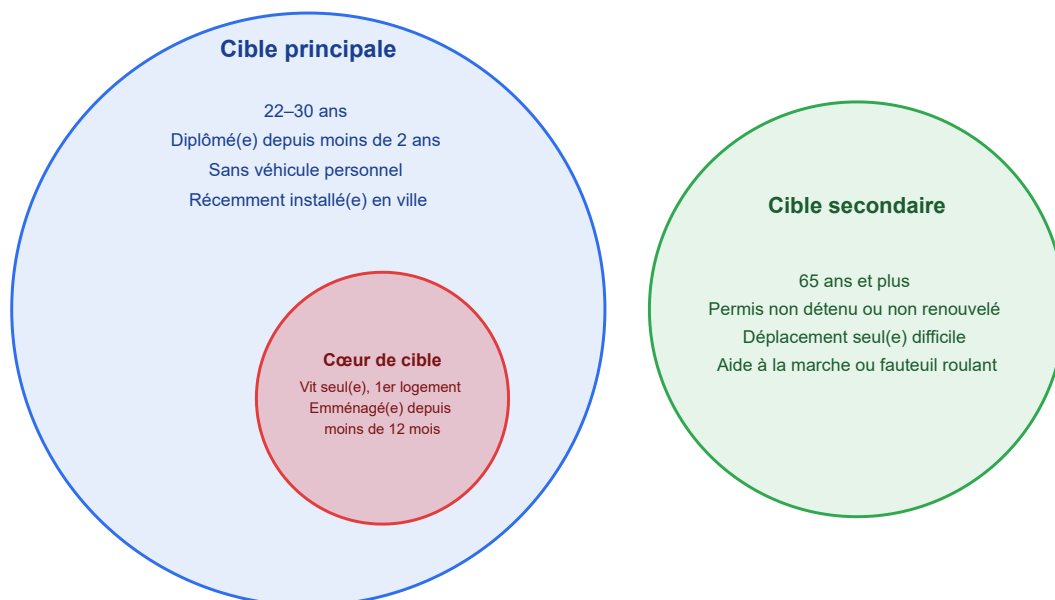
- un planificateur d'itinéraires multimodaux
- un système de réservation unifié
- une IA d'optimisation des trajets en temps réel
- un système de gamification
- un module de covoiturage dynamique
- un suivi de l'empreinte carbone
- des alertes personnalisées
- un tableau de bord citoyen
- un signalement collaboratif des incidents

Si cette liste présente une idée assez juste du périmètre, elle mélange néanmoins des besoins de nature différente : certains relèvent du **cœur de métier de la plateforme** (planifier et réserver un trajet), d'autres sont des **leviers d'engagement** (gamification, tableau de bord) et d'autres encore répondent à des **enjeux de fiabilité du service** (alertes, signalement). Une partie du travail de cadrage a donc consisté à distinguer, derrière la liste de fonctionnalités, les **besoins réels du commanditaire** : réduire la congestion et la pollution, attirer plus de monde vers le vélo, la trottinette, les transports en commun et le covoiturage, et donner à la collectivité une meilleure visibilité sur les usages pour orienter ses futurs investissements en infrastructure.

2.3 Cibles et personas

Cibles

La plateforme ne s'adresse pas de la même façon à tous les habitants de la métropole. Trois niveaux de ciblage sont définis pour orienter les priorités de conception : une cible principale, un cœur de cible plus précis à l'intérieur de celle-ci, et une cible secondaire à ne pas négliger.



Cible principale — les jeunes actifs en début de carrière. Il s'agit des habitants qui viennent de terminer leurs études ou d'obtenir leur premier emploi, entre 22 et 30 ans, diplômés depuis moins de deux ans, sans véhicule personnel et récemment installés en ville. C'est aussi une génération habituée avec les usages numériques et plus spontanément réceptive aux enjeux de mobilité durable, ce qui en fait un point d'entrée naturel pour la plateforme.

Cœur de cible — les jeunes actifs en quête d'autonomie. Au sein de cette cible principale, un sous-groupe se distingue plus particulièrement : celui des jeunes actifs vivant seuls dans leur premier logement, emménagés depuis moins de 12 mois, qui cherchent activement à gagner en autonomie dans cette nouvelle étape de leur vie — ne plus dépendre de leurs proches pour se déplacer, apprivoiser une ville qu'ils connaissent encore mal. Ce besoin d'autonomie correspond directement à ce que propose la plateforme : pouvoir organiser soi-même ses déplacements, sans connaissance préalable du réseau local. C'est ce sous-groupe qui devrait le plus naturellement adopter la plateforme et en devenir un usager régulier.

Cible secondaire — les personnes âgées à mobilité réduite ou limitée. Il s'agit des habitants de 65 ans et plus, qui ne détiennent plus de permis de conduire (ou ne l'ont pas renouvelé), pour qui se déplacer seuls est devenu difficile, et qui ont recours à une aide à la marche ou à un fauteuil roulant. Leurs attentes diffèrent nettement de la cible principale : une plateforme simple à utiliser, des informations fiables sur l'accessibilité des itinéraires, et une certaine tolérance à l'égard des usages numériques moins avancés. Cette cible reste secondaire dans les priorités de conception, mais elle justifie que l'accessibilité (voir [partie 10](#)) ne soit pas traitée comme une simple case à cocher réglementaire.

Personas

Les cibles définies ci-dessus prennent forme à travers deux personas, un pour chacune des cibles les plus déterminantes pour la conception de la plateforme.

PROFIL

- 22 ans, en alternance
- Dernière année d'études
- Premier studio, seul depuis 8 mois
- Pas de véhicule personnel



FRUSTRATIONS

- Jongle entre plusieurs applications (tram, vélos en libre-service)
- Perd du temps à comparer les trajets à la main
- Budget réduit, pas de voiture ni de VTC régulier

OBJECTIFS

- Aller en cours/entreprise sans dépendre de ses proches
- Trouver le trajet le moins cher, vélo + tram, en un coup d'œil
- Gagner en autonomie dans une ville qu'il connaît encore mal

Antoine
22 ans — Alternant, fin d'études
" Je viens d'arriver en ville, je veux me débrouiller seul sans demander à mes proches de me déposer. "

COMPORTEMENTS

- Vérifie son trajet juste avant de partir
- Choisit systématiquement l'option la moins chère
- Teste volontiers un nouvel outil s'il simplifie sa vie

Antoine incarne le cœur de cible : jeune actif en alternance, il a besoin d'autonomie immédiate dans une ville qu'il ne maîtrise pas encore, sans moyen de transport personnel. Sa présence dans le persona justifie que le planificateur d'itinéraires reste rapide et lisible dès la première utilisation, sans phase d'apprentissage.

PROFIL

- 68 ans, retraitée
- Permis rendu il y a 2 ans
- Se déplace avec une canne
- Vit seule en centre-ville



FRUSTRATIONS

- Doit consulter plusieurs sites différents pour vérifier un trajet
- Trottoirs abîmés ou travaux non signalés à l'avance
- Pas assez de bancs pour souffler en cours de trajet

OBJECTIFS

- Aller à ses rendez-vous médicaux en autonomie
- Voir ses proches sans dépendre de quelqu'un
- Savoir à l'avance si le trajet est accessible

Muriel
68 ans — Retraitée, mobilité réduite
" Un trottoir abîmé, encombré ou pas de banc pour souffler, et mon trajet devient une épreuve. "

COMPORTEMENTS

- Prépare son trajet la veille, jamais dans l'urgence
- Privilégie les lignes qu'elle connaît déjà
- Apprécie les outils numériques clairs et bien expliqués

Muriel incarne la cible secondaire : son profil illustre l'importance donnée aux conditions d'accessibilité — un trottoir dégradé/encombré ou l'absence de banc suffit à remettre en cause tout un trajet pour elle, alors que ces obstacles restent invisibles pour la majorité des autres usagers.

Ces deux profils seront repris tout au long du dossier pour justifier certains arbitrages, notamment dans la hiérarchisation des besoins qui suit ([partie 2.5](#)).

2.4 Enjeux métiers et objectifs économiques

Trois enjeux sont identifiés pour structurer le projet du point de vue de la métropole :

- **Enjeu environnemental et de santé publique :**

La réduction du trafic automobile individuel, combinée à une incitation forte vers des modes de déplacement plus doux comme le vélo, la marche ou les transports en commun, s'inscrit directement dans les objectifs de transition écologique portés par la collectivité. Cette évolution des usages est attendue avec des retombées concrètes sur la qualité de l'air et la réduction des nuisances sonores en milieu urbain.

- **Enjeu d'attractivité et de qualité de vie :**

La fluidité des déplacements est un critère de plus en plus déterminant dans le choix d'implantation des habitants et des entreprises. Une plateforme de mobilité efficace constitue un point fort qui valorise l'attractivité de la métropole.

- **Enjeu économique et budgétaire :**

Une meilleure répartition des usagers entre les différents modes de transport permet d'optimiser l'infrastructure existante plutôt que d'investir dans sa seule extension. Les données d'utilisation récupérées par la plateforme serviraient d'atout décisif pour orienter les décisions publiques futures (renforcement d'une ligne de bus, ajout de stations de vélos, etc.).

Ces enjeux justifient que la solution ne soit pas pensée comme un simple outil technique, mais comme un outil au service des décisions de la ville. Ce constat a orienté plusieurs choix fonctionnels détaillés dans les parties suivantes (notamment la priorité donnée au planificateur multimodal et au suivi carbone sur des fonctionnalités plus secondaires comme la gamification).

2.5 Recueil et hiérarchisation des besoins

À partir de la demande client, les besoins sont regroupés en trois niveaux de priorité.

Besoins prioritaires (cœur de la solution) : ce sont les besoins sans lesquels la plateforme n'a pas de valeur d'usage. Ils correspondent aux fonctionnalités obligatoires du cahier des charges : un système de gestion de compte et de profils de mobilité, un planificateur d'itinéraires multimodal avec géolocalisation en temps réel, et l'intégration des données de transport existantes (réseaux GTFS, opérateurs de vélos et trottinettes partagés). C'est exactement ce dont a besoin un usager comme [Antoine](#) dès sa première utilisation : un planificateur fiable, utilisable sans connaissance préalable du réseau local.

Besoins secondaires (ce qui fait sortir du lot) : ils enrichissent l'expérience une fois le socle en place, sans être indispensables à un premier usage. Il s'agit notamment de la réservation unifiée, de l'optimisation d'itinéraires par IA, du covoiturage dynamique, de la gamification, du calculateur d'empreinte carbone et du signallement collaboratif. Le cahier des charges impose d'en implémenter au moins une ; celle retenue pour ce projet est détaillée en [partie 7](#).

Besoins non fonctionnels et transverses : ils ne sont pas visibles directement par l'utilisateur mais conditionnent la qualité et la conformité de la solution. Ils recouvrent la compatibilité PWA (installation, fonctionnement hors ligne partiel), l'ergonomie responsive, la sécurité des données conformément aux standards OWASP, le respect du RGPD pour les données de géolocalisation, l'accessibilité WCAG 2.1 AA pour les personnes

à mobilité réduite, l'éco-conception, et la capacité de la plateforme à fonctionner correctement en situation de connectivité variable (usage mobile en ville). Ces contraintes sont reprises en détail en [partie 10](#), un point qui n'a rien d'accessoire pour un usager comme [Muriel](#), pour qui un trajet mal renseigné peut rendre tout le déplacement très compliqué voir impossible.

2.6 Anticiper les évolutions futures

Un des risques identifiés dès le cadrage est de concevoir une solution figée sur le périmètre initial, alors que le commanditaire est susceptible de faire évoluer ses attentes une fois la plateforme en service : ajout de nouveaux opérateurs de mobilité, extension à d'autres villes de la métropole, ouverture à des partenaires privés (parkings, loueurs de véhicules électriques), ou encore intégration de nouveaux capteurs urbains dans une logique de ville intelligente.

Cette anticipation a orienté deux décisions structurantes qui seront développées plus loin dans le dossier : le choix d'une architecture modulaire où chaque mode de transport est traité comme une source de données interchangeable plutôt que développer à la demande ([partie 4](#)), et l'adoption d'une méthodologie de développement itérative permettant d'intégrer progressivement de nouvelles fonctionnalités sans remettre en cause l'existant ([partie 5](#)).

3. État de l'art et recommandations technologiques

3.1 Benchmark concurrentiel des solutions de mobilité existantes

Avant de faire des choix techniques, il est utile de regarder ce qui existe déjà sur le marché de la mobilité urbaine intelligente (MaaS, *Mobility as a Service*), pour situer UrbanFlow Mobility par rapport aux références du secteur.

Solution	Points forts	Limites au regard du besoin
Whim (MaaS Global, Helsinki)	Pionnier du MaaS, abonnement unique multimodal	Modèle économique par abonnement complexe et coûteux à répliquer pour une métropole
Citymapper	Excellent planificateur multimodal, UX très aboutie	Pas de réservation, pas de paiement intégré, pas de suivi carbone, pas de gamification
Moovit	Très large couverture GTFS à l'international, temps réel fiable	Expérience assez générique, peu de personnalisation par IA
Transit App	Bonne intégration de la micro-mobilité (vélos, trottinettes)	Pas de covoiturage dynamique, ni de tableau de bord citoyen

Aucune de ces solutions ne couvre l'ensemble du périmètre attendu par la métropole (planification multimodale, réservation unifiée, optimisation par IA, suivi carbone, gamification, signalement collaboratif). Cela confirme l'intérêt d'un développement sur-mesure plutôt qu'une solution en marque blanche, tout en s'inspirant de leurs bonnes pratiques (UX de Citymapper, couverture de données de Moovit).

3.2 Standards de données de transport

Le choix des sources de données conditionne fortement l'interopérabilité de la plateforme. Trois standards ouverts, largement adoptés par les opérateurs de transport, ont été retenus plutôt que des intégrations propriétaires :

- **GTFS** (*General Transit Feed Specification*) pour les horaires théoriques des transports en commun, standard de fait dans la quasi-totalité des réseaux urbains.
- **GTFS-Realtime**, extension du même standard, pour les perturbations et positions en temps réel des véhicules.
- **GBFS** (*General Bikeshare Feed Specification*) pour les données de vélos et trottinettes en libre-service, adopté par la plupart des opérateurs de micro-mobilité (Lime, Dott, Tier...).

S'appuyer sur ces standards plutôt que sur des connecteurs propriétaires par opérateur limite la dette technique et facilite l'ajout de nouveaux partenaires à l'avenir, dans la continuité de la logique d'évolutivité posée en [partie 2.6](#). (Pour aller plus loin : origine et adoption de ces standards en [annexe A](#).)

3.3 Moteur de calcul d'itinéraires multimodaux

Le planificateur d'itinéraires est la fonctionnalité cœur de la plateforme. Trois approches ont été comparées :

Option	Description	Avantages	Inconvénients
Développement d'un moteur de routage maison	Coder l'algorithme de calcul d'itinéraires multimodaux	Contrôle total	Complexité algorithmique disproportionnée pour un projet individuel, délai irréaliste
API commerciale (Google Routes API, HERE)	Externaliser le calcul d'itinéraires à un service tiers	Rapide à intégrer, fiable	Coût récurrent au volume d'appels, dépendance à un tiers, moins de finesse sur le multimodal spécifique à la ville
Moteur open-source auto-hébergé (OpenTripPlanner)	Déployer un moteur de routage existant, alimenté par les flux GTFS/GBFS de la métropole	Gratuit, personnalisable, données maîtrisées (cohérent avec le RGPD)	Effort de déploiement et de calibrage initial

Recommandation retenue : OpenTripPlanner (OTP). Ce choix est argumenté par le fait qu'OTP gère nativement le calcul d'itinéraires combinant transport en commun, vélo et marche à partir de flux GTFS, qu'il est reconnu comme référence dans les projets MaaS académiques et municipaux, et qu'il évite une dépendance à un service tiers payant peu compatible avec un budget de collectivité sur le long terme. (*Fonctionnement technique détaillé en [annexe B](#).*)

3.4 Architecture applicative

Le cahier des charges impose une architecture PWA (*Progressive Web App*). Ce choix, indépendamment de son caractère imposé, reste le plus pertinent pour ce projet : une PWA permet de toucher aussi bien les usagers Android qu'iOS depuis une seule base de code, d'être installée sur l'écran d'accueil sans passer par un store, et de fonctionner en partie hors connexion — un point important pour un usage en mobilité avec une couverture réseau variable. Comparée à deux applications natives distinctes (Android et iOS), elle réduit aussi significativement les délais et les coûts de maintenance, ce qui est cohérent avec la logique budgétaire d'une collectivité.

3.5 Architecture applicative : Full Stack intégré ou Frontend/Backend séparés

Approche	Description	Avantages	Inconvénients
Framework full-stack intégré (ex. Next.js, Django avec templates serveur)	Un seul projet gère à la fois l'interface et la logique serveur	Déploiement plus simple, un seul dépôt de code, moins de configuration initiale	Couplage fort entre front et back, plus difficile à faire évoluer indépendamment, peu adapté si un futur client mobile natif doit consommer les mêmes données
Frontend SPA + Backend API REST séparés	Deux projets distincts communiquant via une API	Scalabilité indépendante, API réutilisable (site web, future appli mobile, partenaires tiers), répartition claire des responsabilités, front et back peuvent avancer en parallèle sur des sprints distincts	Complexité de déploiement légèrement supérieure (deux services à maintenir), nécessite de documenter l'API

Recommandation retenue : architecture découplée (frontend SPA + backend API REST). Ce choix répond directement à l'exigence d'interopérabilité du cahier des charges : la même API pourra, à terme, être consommée par d'autres canaux que la PWA (application mobile native, partenaires souhaitant intégrer les données de la plateforme). Il facilite aussi le travail en sprints décrit en [partie 5](#), front et back pouvant être développés en parallèle.

3.6 Benchmark des frameworks Frontend

Framework	Avantages	Inconvénients
React	Écosystème très large, bon support PWA (Workbox, vite-plugin-pwa), bibliothèques cartographiques matures (react-leaflet)	Nécessite des bibliothèques tierces pour certains besoins (gestion d'état, routage)
Vue.js	Courbe d'apprentissage douce, bonne documentation, structure plus intégrée que React	Écosystème plus restreint pour les composants cartographiques spécialisés
Angular	Framework complet "batteries incluses", adapté aux grosses équipes	Verbeux, courbe d'apprentissage plus raide, plus lourd pour une PWA mobile-first
Svelte	Compilation en JavaScript natif, bundles très légers, bonnes performances runtime	Écosystème encore jeune, moins de bibliothèques cartographiques matures, communauté plus réduite

Recommandation retenue : React. Justifié par la maturité de son écosystème PWA, la disponibilité de bibliothèques cartographiques éprouvées, et la taille de sa communauté qui facilite la maintenance du projet dans la durée.

3.7 Benchmark des frameworks Backend

Framework	Langage	Avantages	Inconvénients
NestJS	TypeScript / Node.js	Même langage que le frontend, structure modulaire proche d'Angular, bon support REST, communauté active	Moins mature qu'un framework comme Spring Boot sur de très gros systèmes
Django	Python	Rapide à mettre en œuvre, ORM intégré, écosystème riche en bibliothèques géospatiales (GeoDjango)	Changement de langage par rapport au frontend
Spring Boot	Java	Très robuste, standard en environnement d'entreprise, même langage qu'OpenTripPlanner	Verbeux, courbe d'apprentissage plus longue, peu adapté à un développement rapide en solo
Laravel	PHP	Rapide à prendre en main, bon tooling	Écosystème moins pertinent pour un projet orienté données géospatiales

Recommandation retenue : NestJS (Node.js/TypeScript). Partager le même langage que le frontend réduit les frictions de développement pour un projet porté par une seule personne, sans sacrifier la structure, ni la testabilité du code — un point clé lors de la revue de code.

3.8 Benchmark des bases de données

Base de données	Avantages	Inconvénients
PostgreSQL + PostGIS	Support natif et mature des données géospatiales, modèle relationnel adapté aux liens entre utilisateurs/trajets/réservations, open-source	Nécessite une extension dédiée (PostGIS) à activer et maîtriser
MySQL	Très répandu, bonne performance en lecture	Support géospatial plus limité que PostGIS
MongoDB (NoSQL)	Flexible sur le schéma, adapté à des données peu structurées	Moins adapté aux relations complexes (utilisateur — trajet — réservation), requêtes géospatiales moins abouties que PostGIS
Firebase / Firestore	Mise en place très rapide, hébergement managé	Hébergé par défaut hors Europe, ce qui pose un problème direct de conformité RGPD pour des données de géolocalisation

Recommandation retenue : PostgreSQL + PostGIS. Seule option combinant nativement un modèle relationnel adapté aux données du projet et un support géospatial de référence, sans compromis sur la localisation d'hébergement en Europe.

3.9 Benchmark des hébergeurs cloud

Hébergeur	Avantages	Inconvénients
AWS / Google Cloud / Azure	Offre la plus large, tous les services disponibles, forte scalabilité	Hébergement par défaut hors UE pour une partie des services, question de souveraineté des données pour une collectivité française, empreinte carbone moins documentée publiquement
Scaleway	Hébergeur français, data centers en France, communication sur son efficacité énergétique, tarification prévisible	Offre de services managés moins large qu'AWS/GCP
OVHcloud	Hébergeur français/européen, très implanté auprès des collectivités, engagements environnementaux (refroidissement à eau)	Interface et outillage parfois moins aboutis que les géants américains

Recommandation retenue : Scaleway ou OVHcloud. Ce choix répond directement à deux contraintes du cahier des charges : la RGPD (données hébergées en France/UE) et l'éco-conception (engagements environnementaux documentés), tout en restant cohérent avec le profil du commanditaire — une collectivité publique française.

Mise en œuvre effective : OVHcloud. Le prototype est déployé sur un serveur OVHcloud (VPS, data center en France), avec **Caddy** en reverse proxy sur l'hôte (terminaison TLS, en-têtes de sécurité, routage `/api` vers le backend) et les services applicatifs (backend NestJS, PostgreSQL/PostGIS, OpenTripPlanner, géocodeur, relai mail) en conteneurs Docker Compose. Le déploiement est automatisé : à chaque fusion sur la branche principale, la CI reconstruit le frontend, synchronise les fichiers statiques et redémarre les conteneurs du serveur (détail du pipeline en [partie 5.2](#)).

3.10 Stack technique retenue — synthèse

Brique	Choix retenu	Détaillé en
Architecture	Frontend SPA + Backend API REST	3.5
Frontend	React + Vite, Workbox pour le service worker	3.6
Backend	NestJS (Node.js/TypeScript)	3.7
Base de données	PostgreSQL + extension PostGIS	3.8
Moteur de routage	OpenTripPlanner	3.3
Hébergement	OVHcloud (VPS en France), Caddy en reverse proxy, conteneurs Docker Compose	3.9
Authentification	JWT avec refresh tokens à rotation, mots de passe hachés (bcrypt)	Annexe C

3.11 Synthèse des arbitrages

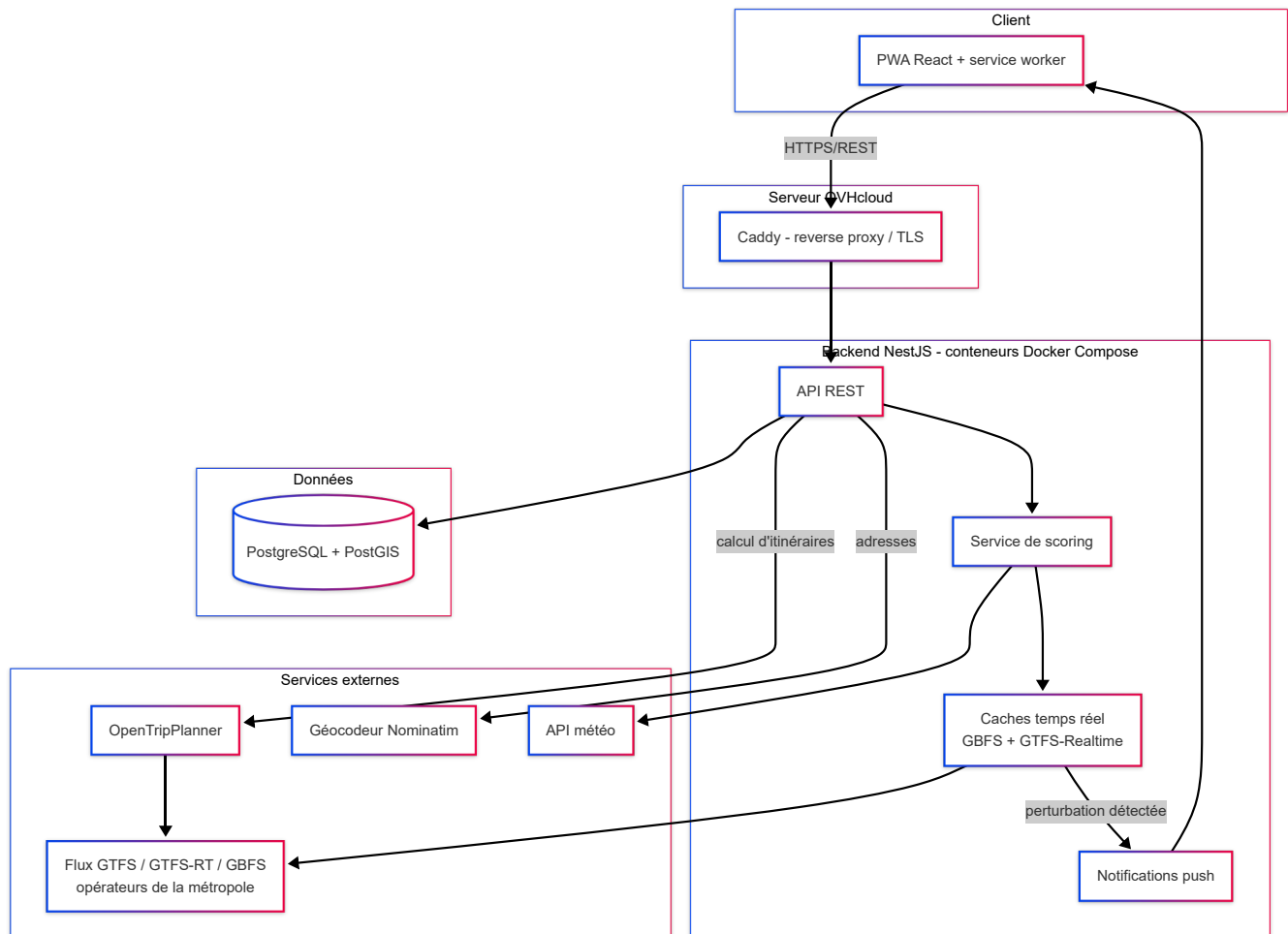
Arbitrage	Coût	Délai	Performance	Pérennité
Architecture découplée plutôt que full-stack intégré	Coût de développement équivalent	Sprints front/back en parallèle	Scalabilité indépendante des deux couches	API réutilisable pour de futurs canaux (mobile, partenaires)
React plutôt que Vue/Angular/Svelte	Pas d'écart significatif	Écosystème mature, prise en main rapide	Réactivité suffisante pour un usage mobile	Grande communauté, maintenabilité assurée
NestJS plutôt que Django/Spring Boot/Laravel	Pas d'écart significatif	Même langage que le frontend, un seul environnement à maîtriser	Performances adaptées à l'échelle du projet	Structure modulaire facilitant l'évolution du code
PostgreSQL/PostGIS plutôt que MySQL/MongoDB/Firebase	Solution open-source, pas de coût de licence	Mise en place légèrement plus longue (extension PostGIS)	Requêtes géospatiales natives et performantes	Solution mature, pas de dépendance à un hébergeur unique
OpenTripPlanner plutôt qu'API commerciale	Pas de coût récurrent à l'usage, coût d'hébergement maîtrisé	Déploiement initial plus long, mais pas de dépendance externe bloquante	Performances suffisantes pour l'échelle d'une métropole, calibrable	Solution open-source active, pérenne indépendamment d'un fournisseur tiers
PWA plutôt que natif double plateforme	Un seul développement au lieu de deux	Délai divisé par rapport celui de deux applications natives	Légèrement en retrait sur l'accès aux capteurs natifs, acceptable pour ce périmètre	Une seule base de code à maintenir dans le temps
Hébergement européen plutôt que cloud américain généraliste	Tarification comparable	Délai de mise en place équivalent	Performance équivalente pour un usage régional	Conformité RGPD native, réduit le risque réglementaire à long terme

Ce tableau synthétise les arbitrages clés du projet et les principes qui les sous-tendent : préférer des solutions ouvertes, interopérables et hébergées en Europe plutôt que des solutions propriétaires plus rapides à mettre en œuvre mais plus risquées sur la durée.

4. Architecture globale et spécifications fonctionnelles

4.1 Vue d'ensemble de l'architecture

L'architecture retenue reprend les choix argumentés en partie 3 : une PWA côté client, une API REST côté serveur, et une séparation nette entre les couches applicatives pour préserver l'évolutivité de la solution.



Ce schéma sépare l'interface utilisateur, la logique métier et les données, avec le moteur de routage et les flux opérateurs traités comme des services externes interchangeableables, cohérent avec la logique d'évolutivité posée en [partie 2.6](#). Le service de scoring ([partie 7](#)), les caches temps réel et le canal de notifications sont des modules du backend, ajoutés au fil des itérations sans remettre en cause cette structure en couches.

4.2 Description des composants

Composant	Rôle	Détaillé en
PWA (React)	Interface utilisateur, planification et suivi des trajets, affichage cartographique, installation sur l'écran d'accueil	3.6
Caddy (reverse proxy)	Terminaison TLS, en-têtes de sécurité HTTP, routage <code>/api</code> vers le backend et service des fichiers statiques du frontend	3.9
API REST (NestJS)	Point d'entrée unique du backend : comptes et profils, recherche et historique de trajets, trajet suivi, abonnements de notification	3.7
Service de scoring	Classement pondéré des itinéraires renvoyés par OpenTripPlanner selon la météo, les perturbations et le profil de l'utilisateur	7.3
Caches temps réel (GBFS, GTFS-Realtime)	Disponibilité des vélos/trottinettes en station et perturbations en cours, rafraîchis en tâche de fond et servis depuis la mémoire	7.3
Notifications push (Web Push / VAPID)	Alerte l'utilisateur qui suit un trajet dès qu'une perturbation est détectée sur sa ligne	7.2
Base de données (PostgreSQL/PostGIS)	Comptes, profils de mobilité, historique et trajets suivis, données géospatiales ; coordonnées et libellés d'adresse chiffrés au repos	3.8
OpenTripPlanner	Calcul des itinéraires multimodaux à partir des flux GTFS/GBFS	3.3
Géocodeur (Nominatim)	Résolution des adresses saisies en coordonnées, sur les données OpenStreetMap de la métropole	—

La **réservation unifiée** et le **calculateur d'empreinte carbone**, évoqués dans la demande initiale ([partie 2.2](#)), ne font pas partie du périmètre implémenté : le cahier des charges n'impose qu'une fonctionnalité au choix ([partie 2.5](#)), et celle retenue est le classement personnalisé ([partie 7](#)). Ces deux briques sont remplacées comme évolutions en [partie 11.1](#).

4.3 Nomenclature et conventions

Pour garder une base de code lisible et homogène, une convention de nommage a été fixée dès le départ :

Élément	Convention	Exemple
Endpoints API REST	Pluriel, kebab-case, verbes HTTP standards	<code>GET /trips</code> , <code>POST /reservations</code>
Tables de base de données	Pluriel, snake_case	<code>users</code> , <code>trip_segments</code> , <code>mobility_profiles</code>
Composants React	PascalCase	<code>TripPlanner</code> , <code>MobilityDashboard</code>
Services NestJS	Suffixe explicite du rôle	<code>TripService</code> , <code>ReservationService</code>

Cette homogénéité facilite la lecture du code lors des revues et réduit le risque d'ambiguïté entre les différentes couches de l'application.

4.4 Spécifications fonctionnelles des modules principaux

Module	Fonctionnalités couvertes	Type
Comptes et profils	Inscription, connexion, gestion du profil de mobilité (préférences de transport)	Obligatoire (F1)
Planification d'itinéraires	Recherche multimodale, géolocalisation en temps réel, affichage cartographique	Obligatoire (F2)
Intégration transport	Connexion aux flux GTFS et GBFS des opérateurs de la métropole	Obligatoire (F3)
Fonctionnalité complémentaire	Détaillée en partie 7	Au choix

Le profil de mobilité (F1) et le planificateur (F2) couvrent directement les besoins des deux personas présentés en [partie 2.3](#) : le premier permet de préciser des préférences comme l'évitement des escaliers ou un mode de transport prioritaire, utile pour un usager comme Muriel, tandis que le second doit rester utilisable sans apprentissage préalable du réseau, condition posée par le profil d'Antoine.

Les préférences de mode du profil portent sur les modes réellement produits par le moteur de routage dans cette version (marche, vélo, bus, tram, métro, train régional). Le vélo/trottinette en libre-service et le covoiturage restent traités comme des **sources de données d'opérateurs externes**, intégrables via un flux GBFS/GTFS conforme sans modification du code applicatif ([partie 4.5](#)) : le libre-service en station est déjà affiché sur la carte (disponibilité temps réel), son intégration au calcul d'itinéraire multimodal et le covoiturage dynamique relèvent des évolutions listées en [partie 11.1](#).

Le détail fonctionnel et technique complet de la fonctionnalité complémentaire retenue est traité spécifiquement en [partie 7](#), pour éviter les redites : cette partie 4 se concentre sur l'architecture d'ensemble et les modules du socle commun.

4.5 Évolutivité et maintenabilité

Trois choix structurants garantissent que la plateforme peut évoluer sans réécriture majeure :

- **Découplage frontend/backend** ([partie 3.5](#)) : l'API pourra être réutilisée par un futur client mobile natif ou des partenaires, sans toucher au backend.
- **Moteur de routage en service externe** : chaque nouvel opérateur de mobilité s'intègre en publiant un flux GTFS ou GBFS conforme, sans modification du code applicatif — voir [partie 2.6](#).
- **Base de données relationnelle avec extension géospatiale** ([partie 3.8](#)) : le modèle de données peut accueillir de nouveaux types de trajets ou de réservations par simple ajout de tables, sans remise en cause du schéma existant.

Cette architecture est pensée pour rester lisible et maintenable dans la durée, condition posée par le cahier des charges pour la partie spécifications.

5. Méthodologie de gestion de projet

5.1 Approche méthodologique retenue

Le cahier des charges impose une approche itérative du cycle de vie de la solution. Une approche Agile inspirée de Scrum a été retenue, adaptée au contexte d'un projet mené par une seule personne : le développement est organisé en sprints de deux semaines, chacun se terminant par une revue fonctionnelle et une rétrospective, plutôt qu'un développement en un seul bloc jusqu'à la livraison finale.

Ce choix permet d'intégrer progressivement les fonctionnalités obligatoires puis la fonctionnalité complémentaire (voir [partie 4.4](#)), de détecter les points de friction tôt, et de garder une marge d'ajustement jusqu'à la fin du projet plutôt que de découvrir des problèmes structurants en toute fin de développement.

5.2 Environnement et outils de travail

Besoin	Outil retenu	Usage
Gestion de version	Git / GitHub	Historique du code, branches par fonctionnalité
Suivi de l'avancement	GitHub Projects (tableau Kanban)	Backlog, sprint en cours, colonnes À faire / En cours / En revue / Terminé
Intégration et déploiement continus	GitHub Actions	Tests, lint, audit WCAG et scan de secrets à chaque push ; déploiement automatique sur fusion vers la branche principale
Tests d'API	Postman	Vérification manuelle et automatisée des endpoints REST
Tests unitaires	Jest (backend), Vitest (frontend)	Tests du backend NestJS et des composants React
Tests d'accessibilité end-to-end	Playwright + axe-core	Audit WCAG 2.1 AA en conditions réelles, exécuté automatiquement en CI
Scan de secrets	gitleaks	Détection de secrets commités, sur l'historique complet, à chaque push
Documentation	README + dossier docs/ versionné dans le dépôt	Spécifications, décisions d'architecture, plans et rétrospectives de sprint
Design d'interface	Figma	Maquettes des écrans avant développement frontend
Assistance au développement	Assistant IA de codage	Accélération de l'écriture de code sous supervision directe, relecture et validation systématiques avant intégration

Ces outils ont été choisis pour leur gratuité (ou leur plan gratuit suffisant à l'échelle du projet), leur bonne intégration avec la stack retenue en partie 3, et leur usage répandu dans l'industrie, ce qui limite le risque de dépendre d'un outil peu documenté ou abandonné.

Pipeline d'intégration et de déploiement. Chaque push déclenche, en parallèle : lint et tests unitaires des deux sous-projets, audit d'accessibilité WCAG de bout en bout (Playwright + axe-core, sur une pile applicative montée pour l'occasion), et scan de secrets sur l'historique git complet. Toute fusion vers la branche principale enchaîne, si ces vérifications passent, sur un déploiement automatique du serveur OVHcloud (reconstruction du frontend, synchronisation des fichiers statiques, redémarrage des conteneurs). Deux garde-fous ont été ajoutés en cours de projet à la suite d'incidents réels (voir [partie 9.4](#)) : une file d'attente empêchant deux déploiements concurrents de se disputer les mêmes conteneurs, et une vérification qui compare les variables d'environnement du serveur à celles attendues par le code et alerte en cas d'écart.

Le recours à un assistant IA de codage mérite une précision : il est utilisé comme un outil d'accélération de la production de code, au même titre qu'un autocomplète avancé ou qu'un pair-programmeur, mais chaque ligne produite est relue, comprise et validée avant d'être intégrée. La responsabilité du code, sa correction et sa justification restent entièrement assumées dans le cadre du projet, y compris lors des revues de code en face-à-face.

5.3 Rôles et responsabilités

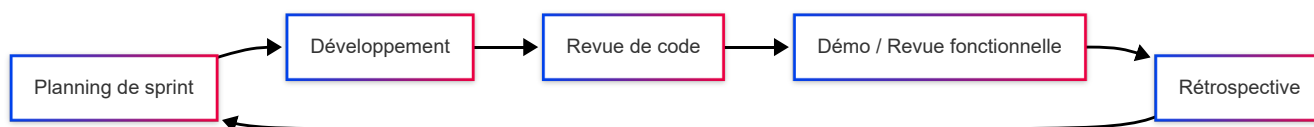
Le projet étant mené individuellement, chaque rôle habituellement porté par un membre d'équipe distinct est ici endossé en interne, sans répartition entre plusieurs personnes. Les distinguer reste utile : cela clarifie, à chaque instant du projet, sous quelle casquette une décision est prise, et permet de présenter clairement le fonctionnement du projet aux différentes parties prenantes (commanditaire, jury, futurs collaborateurs).

Rôle	Responsabilités
Product Owner	Priorisation du backlog, arbitrages fonctionnels, relation avec le commanditaire
Développeur Frontend	Implémentation de la PWA React, intégration des maquettes Figma
Développeur Backend	Implémentation de l'API NestJS, intégration d'OpenTripPlanner et des flux GTFS/GBFS
Testeur / QA	Écriture et exécution des tests, revue de code
Référent qualité et sécurité	Vérification des critères OWASP, RGPD, accessibilité (voir partie 10)

L'ensemble de ces rôles est assumé par une seule et même personne dans le cadre de ce projet individuel ; le découpage ci-dessus sert avant tout à structurer le raisonnement et à faciliter le dialogue avec le commanditaire sur qui fait quoi et à quel moment.

5.4 Déroulement d'un cycle d'itération

Chaque sprint de deux semaines suit le même déroulé, présenté ci-dessous.



- **Planning de sprint** : sélection des tickets du backlog à traiter, en fonction de leur priorité et de leur dépendance avec les fonctionnalités déjà livrées.
- **Développement** : implémentation par petites unités testables, commits réguliers sur des branches dédiées.

- **Revue de code** : relecture systématique avant fusion sur la branche principale, avec vérification du respect des conventions posées en [partie 4.3](#).
- **Démo / revue fonctionnelle** : vérification que les fonctionnalités livrées répondent aux besoins exprimés en [partie 2](#).
- **Rétrospective** : bilan du sprint, ajustements pour le sprint suivant — cette étape est reprise et approfondie en [partie 6](#) dans une logique d'amélioration continue.

Ce cycle répété tout au long du projet permet d'assurer le bon déroulement des itérations de production demandé par le cahier des charges, tout en gardant une trace claire des décisions prises à chaque étape.

6. Démarche qualité et amélioration continue

6.1 Indicateurs de qualité suivis

La qualité n'est pas vérifiée une seule fois en fin de projet : elle est suivie tout au long du développement à travers quelques indicateurs simples, choisis pour rester réalistes à l'échelle d'un développeur unique plutôt que pour reproduire un dispositif de suivi pensé pour une grande équipe.

Indicateur	Outil	Fréquence
Couverture de tests	Jest (backend), Vitest (frontend)	À chaque exécution de la CI
Conformité de style et détection d'erreurs statiques	ESLint / Prettier	À chaque commit (hook local) et à chaque push
Statut des tests automatisés	GitHub Actions	À chaque push
Anomalies remontées en test manuel	Postman + suivi GitHub Projects	À chaque sprint
Respect des critères transverses (sécurité, accessibilité, performance)	Revue manuelle ciblée	Avant chaque mise en production

Ces indicateurs ne remplacent pas la relecture humaine du code, mais donnent un premier résumé objectif avant toute revue de code, et permettent de repérer une régression dès qu'elle apparaît plutôt qu'au moment de la livraison finale.

6.2 Démarche d'amélioration continue

La démarche qualité s'appuie sur la logique du cycle DMAIC (Définir, Mesurer, Analyser, Améliorer, Contrôler), habituellement utilisée en gestion de la qualité industrielle et transposée ici à l'échelle d'un sprint de développement : chaque itération part d'un objectif défini, s'appuie sur les indicateurs présentés en [partie 6.1](#) pour mesurer l'état du produit, analyse les écarts constatés, met en œuvre des actions correctives ciblées, puis vérifie que ces actions tiennent dans la durée avant de passer au sprint suivant.

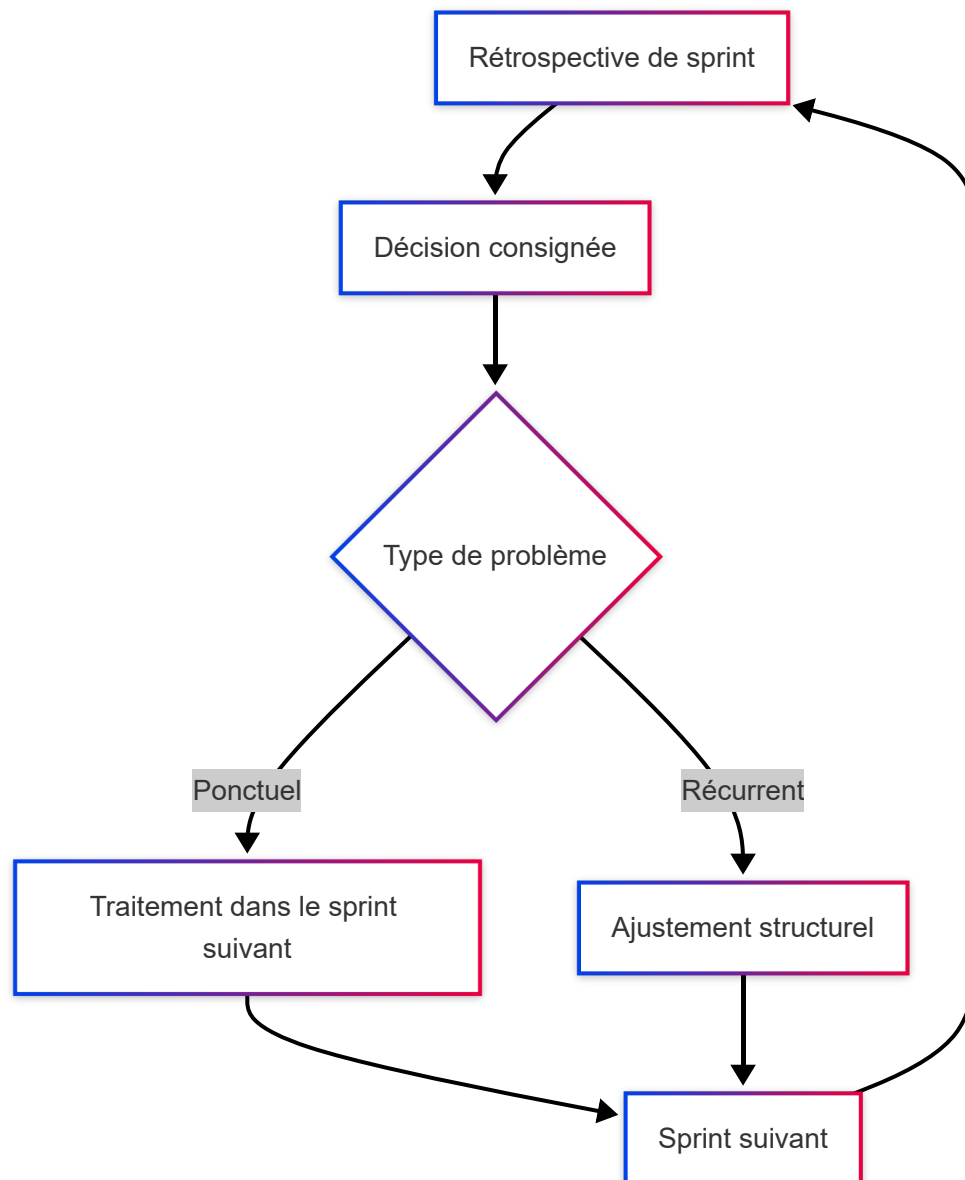


Cette logique rejoint l'esprit du Kaizen : privilégier une série de petites améliorations continues, décidées à chaque rétrospective, plutôt que d'attendre une refonte lourde en fin de projet. Elle s'articule directement avec le déroulement de sprint déjà posé en [partie 5.4](#), où la rétrospective sert justement de point d'entrée à ce cycle : les constats de fin de sprint deviennent les objectifs du sprint suivant.

6.3 Boucle de capitalisation

Pour que chaque cycle profite réellement au suivant, les décisions prises en rétrospective sont consignées, plutôt que de rester une remarque orale vite oubliée. Concrètement, l'historique des décisions du projet est **consultable**

en continu à travers quatre traces croisées, versionnées dans le dépôt : les messages de commits et les *pull requests* (une par unité de travail, avec sa justification), les tickets GitHub fermés (127 à ce jour, reliés aux commits qui les traitent), les plans de sprint ([docs/sprints/sprint-N-plan.md](#) , qui figent l'ordre et le raisonnement décidés en séance) et les rétrospectives de fin de sprint ([docs/sprints/sprint-N-retro.md](#)). Toute décision d'architecture argumentée renvoie à ce dossier ; toute convention qui évolue est répercutée dans [CLAUDE.md](#) , le fichier de contexte de conception à la racine du dépôt.



Deux cas sont distingués :

- une anomalie ponctuelle, traitée directement dans le sprint suivant et documentée dans la logique détaillée en [partie 9](#) ;
- un problème récurrent (même type d'erreur qui revient, convention mal respectée), qui déclenche un ajustement plus structurel — évolution d'une règle de lint, ajout d'un test de non-régression, mise à jour d'une convention posée en [partie 4.3](#).

Cette distinction évite de traiter chaque problème comme un cas isolé et permet à la démarche qualité de rester utile sur la durée du projet plutôt que de se limiter à un contrôle final avant livraison.

6.4 Bilan quantitatif de la démarche

Au terme du projet, la démarche qualité se traduit par les indicateurs suivants (état à la clôture, dépôt public sur GitHub) :

Indicateur	Valeur
Durée du projet	7 semaines, 5 sprints de 2 semaines
Tickets fermés / ouverts	127 / 2
<i>Pull requests</i> fusionnées	151 (une par unité de travail, chacune passée en revue avant fusion)
Tests automatisés	~272 tests backend (Jest, 35 suites), ~345 tests frontend (Vitest, 36 suites)
Tests end-to-end	2 suites API (dont une de non-régression sur le rate limiting), 1 suite d'audit WCAG 2.1 AA (Playwright + axe-core) exécutée à chaque push
Audits transverses	Audit OWASP en deux passes (cadrage initial puis audit dédié approfondi), audit d'accessibilité WCAG automatisé en CI, scan de secrets sur l'historique git
Corrections issues des tests en conditions réelles	Plusieurs constats remontés par des revues manuelles de bout en bout ont donné lieu à des sprints de correction dédiés (voir partie 9.4)

Ces chiffres ne valent pas pour eux-mêmes : ils traduisent surtout que la vérification a été **continue** (à chaque push, à chaque fin de sprint) plutôt que concentrée dans un contrôle final, et que chaque anomalie détectée a laissé une trace exploitable : un ticket, une *pull request*, souvent un test de non-régression.

7. Spécifications détaillées d'une fonctionnalité clé

7.1 Présentation et objectifs de la fonctionnalité

La fonctionnalité complémentaire choisie est l'**optimisation d'itinéraires par IA en temps réel**, reprise telle quelle de la liste des fonctionnalités au choix du cahier des charges et identifiée comme besoin secondaire en [partie 2.5](#). La [partie 7.3](#) précise ce que recouvre ici le terme d'IA : une aide à la décision par pondération explicite, pas un modèle d'apprentissage. Elle vient s'ajouter au planificateur multimodal de base ([partie 3.3](#)) plutôt que le remplacer : OpenTripPlanner reste responsable du calcul des itinéraires possibles à partir des flux GTFS/GBFS, et cette fonctionnalité ajoute une couche d'ajustement au-dessus, capable de tenir compte de conditions changeantes (perturbations, météo) et des préférences propres à chaque usager.

L'objectif n'est pas de calculer "LE" meilleur trajet dans l'absolu, mais le trajet le plus pertinent pour une personne donnée, à un instant donné. C'est exactement ce que recherche un usager comme [Antoine](#) : un trajet fiable proposé sans qu'il ait à comparer lui-même plusieurs options.

7.2 Spécifications fonctionnelles

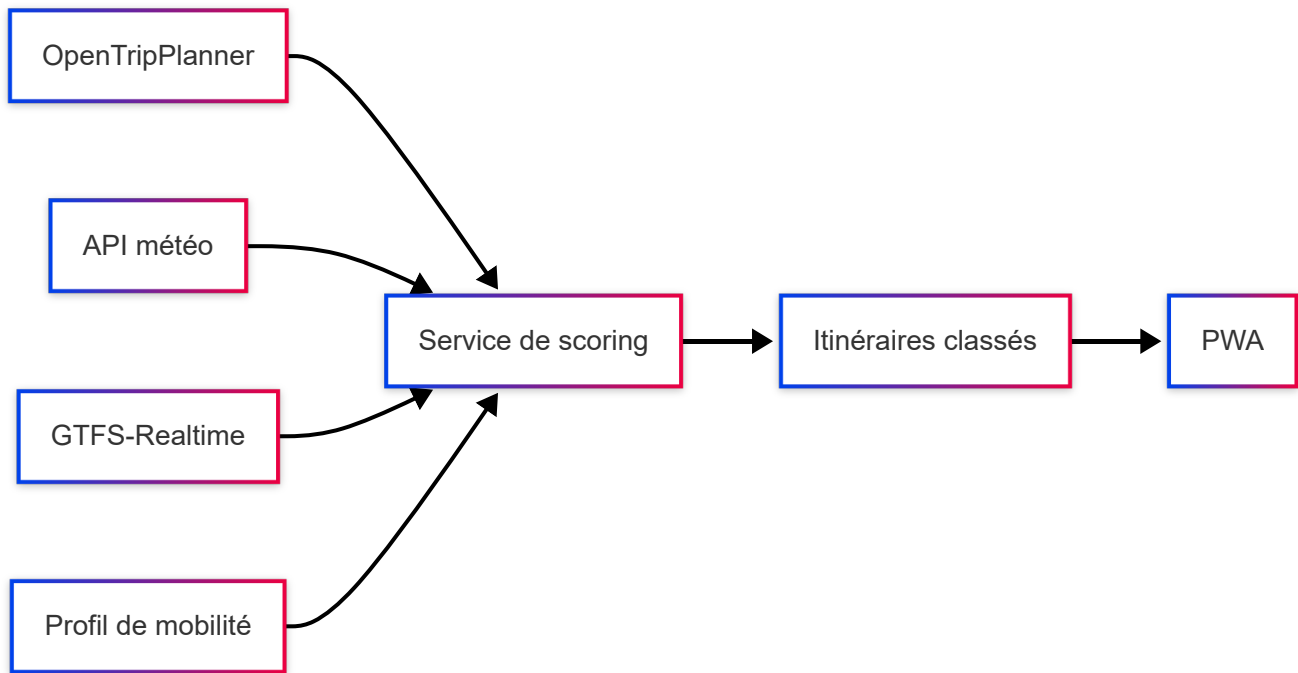
Deux cas d'usage principaux sont couverts :

Recherche avec classement personnalisé. Lorsqu'un usager demande un itinéraire, plusieurs propositions issues du moteur de routage sont comparées et classées selon plusieurs critères, chacun affecté d'un poids explicite et modifiable, plutôt que par le seul temps de trajet théorique :

Critère	Exemple d'impact
Temps de trajet estimé	Critère de base, déjà fourni par le moteur de routage
Nombre de correspondances	Pénalise les trajets avec beaucoup de changements, sauf si l'usager les tolère
Conditions météo en cours	Déprioritise un trajet à vélo en cas de forte pluie annoncée
Perturbations en cours	Déprioritise une ligne signalée en incident via GTFS-Realtime
Préférences enregistrées	Priorise ou évite un mode de transport selon le profil de mobilité (partie 4.4)

Ajustement en cours de trajet. Si un incident est signalé sur la ligne d'un trajet suivi après le départ (ligne interrompue, retard important), le backend relance un calcul d'itinéraire tenant compte de la perturbation et envoie une notification push ; à l'ouverture de l'application, l'usager retrouve des itinéraires réévalués, plutôt que de découvrir le problème une fois bloqué sur place. C'est la situation vécue par [Muriel](#) lors d'un imprévu sur son trajet.

7.3 Spécifications techniques



Ce que fait, et ne fait pas, cette IA. OpenTripPlanner conserve le calcul d'itinéraires proprement dit : il explore le graphe des transports (algorithmes de plus court chemin de type Dijkstra, et RAPTOR pour le volet transports en commun) et renvoie plusieurs itinéraires candidats. La brique ajoutée ici n'est pas un moteur de graphe supplémentaire, ni un modèle d'apprentissage : c'est une **fonction de coût par somme pondérée** appliquée aux itinéraires déjà calculés. Chaque critère du tableau ci-dessus reçoit un poids explicite, défini dans un unique fichier de configuration ([scoring-weights.const.ts](#)). Le coût total de chaque itinéraire est la combinaison linéaire de ces critères, et le classement est le tri par coût croissant. Cette approche relève de l'**analyse multicritère d'aide à la décision** (*weighted sum model*), pas du *machine learning*.

Ce choix est délibéré sur un projet individuel. Le résultat est **justifiable** (on peut expliquer au tableau pourquoi tel trajet passe devant tel autre), **débogable**, **ajustable** sans ré-entraînement, et il ne dépend d'aucun volume de données d'entraînement qui n'existe pas encore. Le terme d'IA est employé ici au sens large d'aide à la décision automatisée ; la trajectoire vers un modèle prédictif entraîné sur l'usage réel est traitée en [partie 7.4](#).

Les poids par défaut sont calibrés manuellement. L'utilisateur les ajuste indirectement via les **préférences de son profil de mobilité** : cocher un mode de transport préféré ajoute un bonus aux itinéraires qui l'empruntent, cocher la limitation des correspondances ou de la distance de marche augmente la pénalité correspondante. Ce ne sont pas des curseurs numériques mais des préférences catégorielles, qui restent le levier de personnalisation du classement. Le réajustement en cours de trajet s'appuie sur un abonnement aux mises à jour GTFS-Realtime : une perturbation détectée sur la ligne empruntée déclenche un nouveau calcul et une notification push vers la PWA.

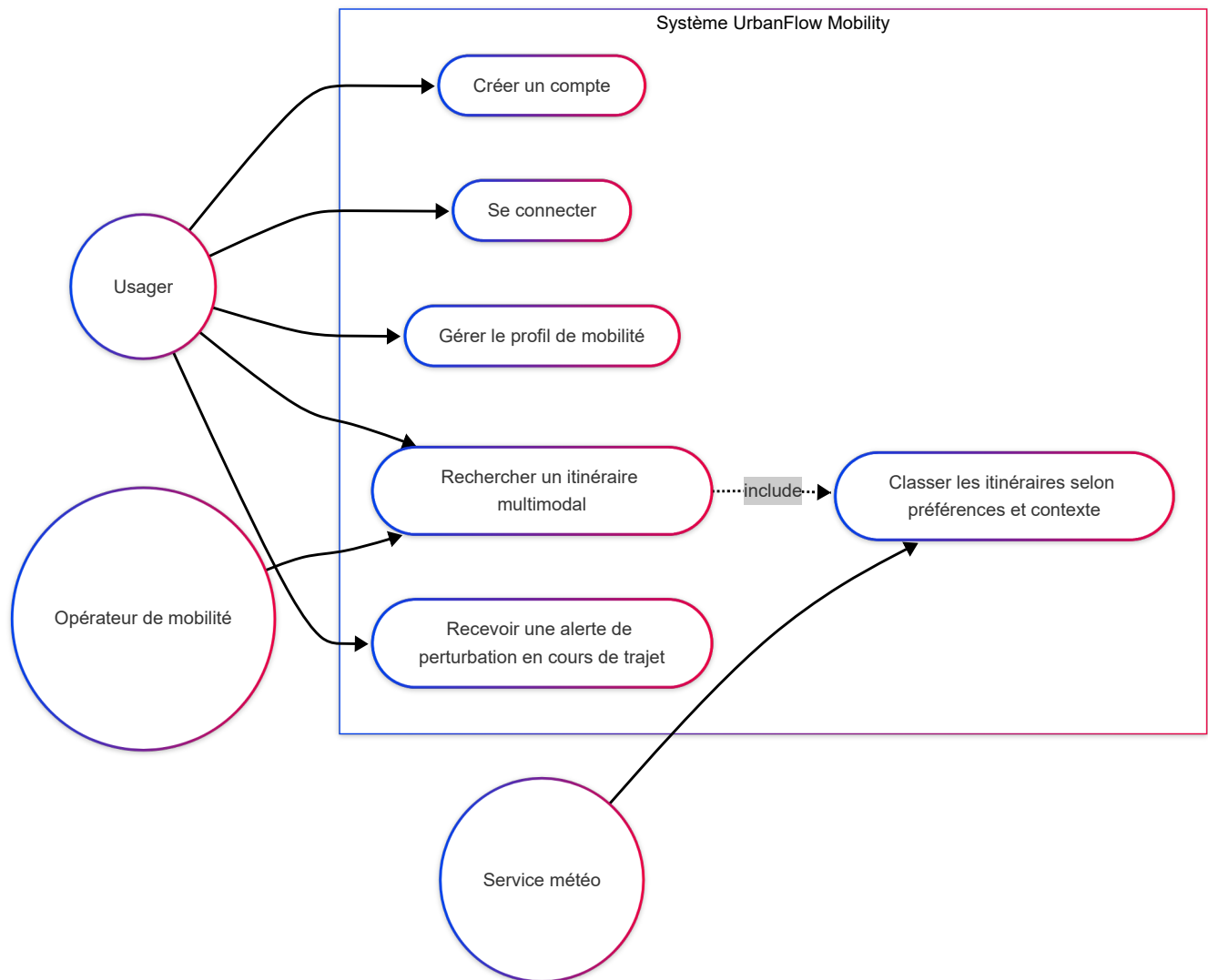
7.4 Limites et évolutions possibles

Cette première version ne prédit pas le trafic à l'avance : elle réagit aux données disponibles au moment de la requête GTFS-Realtime plutôt que d'anticiper une perturbation avant qu'elle ne soit signalée. Un modèle prédictif plus poussé, entraîné sur un historique d'usage réel de la plateforme, est une évolution envisageable une fois suffisamment de données collectées — mais hors de portée réaliste pour la version initiale du projet. La qualité du classement dépend aussi directement de la fraîcheur des données météo et trafic fournies par les services tiers, ce qui en fait une dépendance externe à surveiller plutôt qu'une limite propre à la conception retenue.

8. Diagrammes UML

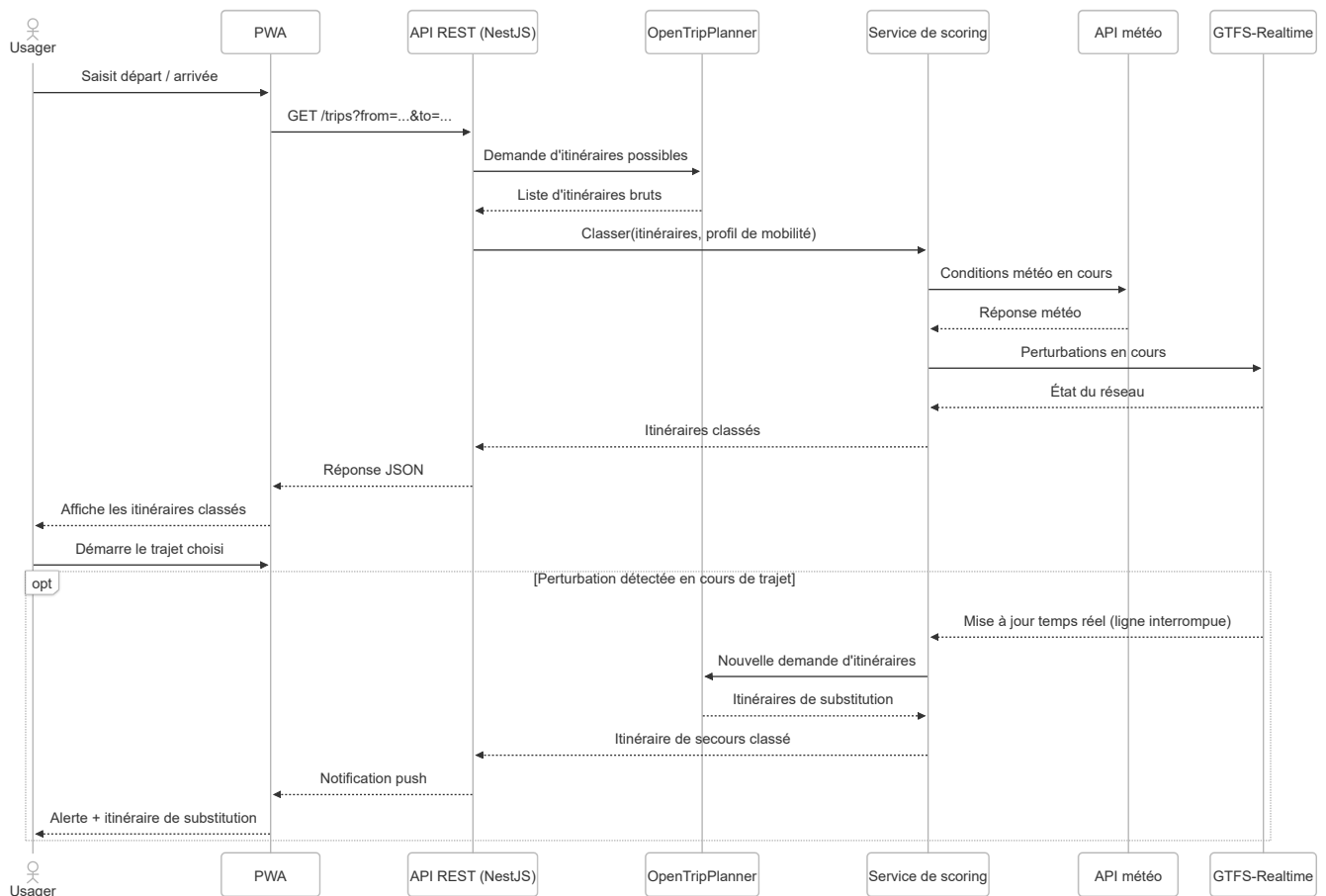
Trois diagrammes UML complémentaires sont présentés ci-dessous, modélisant chacun un angle différent du même parcours central de la plateforme : la recherche d'itinéraire avec classement personnalisé, spécifiée en [partie 7](#).

8.1 Diagramme de cas d'utilisation



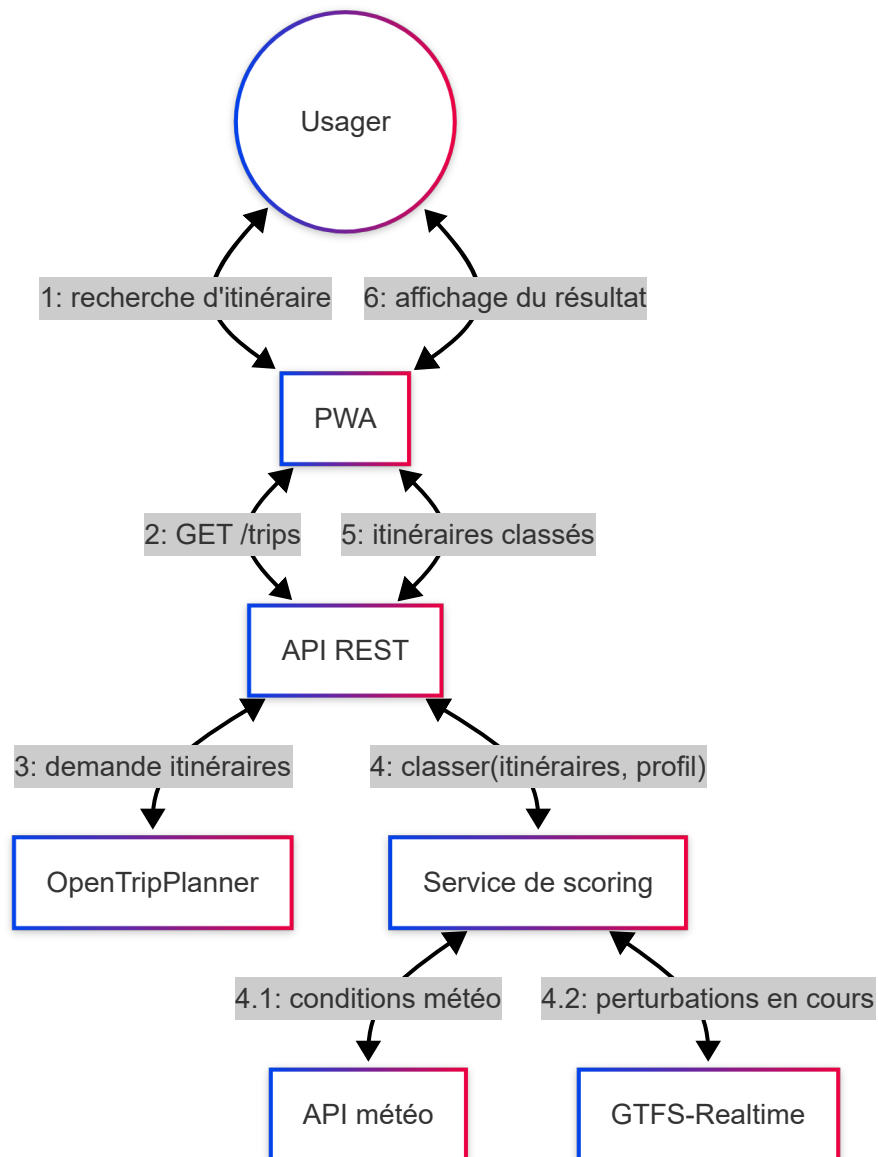
L'acteur principal est l'utilisateur de la plateforme, qu'il s'agisse d'un profil comme Antoine ou comme Muriel ([partie 2.3](#)) : il crée un compte, se connecte, renseigne son profil de mobilité, puis recherche un itinéraire. Deux acteurs secondaires interviennent en soutien sans jamais interagir directement avec l'utilisateur : l'opérateur de mobilité, qui alimente la recherche d'itinéraire via ses flux GTFS/GBFS ([partie 3.2](#)), et le service météo, mobilisé uniquement par le cas d'usage de classement des itinéraires. Ce dernier cas d'usage est relié à la recherche d'itinéraire par une relation d'inclusion : on ne classe jamais un itinéraire sans qu'une recherche ait d'abord été effectuée — cette fonctionnalité correspond à celle détaillée en [partie 7](#). Le cas d'usage "recevoir une alerte de perturbation" reste indépendant : il peut se déclencher à tout moment après le départ, sans action supplémentaire de l'utilisateur.

8.2 Diagramme de séquence



Ce diagramme couvre les deux cas d'usage définis en [partie 7.2](#). La première moitié détaille le scénario nominal de recherche : l'utilisateur ne dialogue qu'avec la PWA, tandis que l'API orchestre l'appel à OpenTripPlanner ([partie 3.3](#)) puis délègue le classement au service de scoring, qui interroge à son tour la météo et GTFS-Realtime avant de renvoyer une liste ordonnée — un enchaînement cohérent avec l'architecture en couches de la [partie 4.1](#). Le bloc optionnel qui suit illustre le second cas d'usage : si une perturbation est détectée après le départ, le service de scoring déclenche automatiquement un recalcul et une notification push, sans action de l'utilisateur — le scénario concret vécu par [Muriel](#) lors d'un imprévu sur son trajet.

8.3 Diagramme de communication



Contrairement au diagramme de séquence, ce schéma met en avant les liens structurels entre les objets impliqués plutôt que leur enchaînement chronologique — chaque flèche représente un canal de communication, numéroté dans l'ordre des échanges. On y retrouve les mêmes composants que dans le diagramme de séquence de la [partie 8.2](#) et l'architecture posée en [partie 4.1](#) : la PWA ne communique qu'avec l'API REST, qui centralise les appels vers OpenTripPlanner et le service de scoring, ce dernier étant le seul point de contact avec les sources externes (météo, GTFS-Realtime). Les messages 4.1 et 4.2, imbriqués sous le message 4, montrent que ces deux appels sont déclenchés par le classement des itinéraires et non par l'API directement — une lecture que le diagramme de séquence rend moins immédiate.

9. Gestion des bogues et qualité de code

La gestion des bogues n'est pas traitée comme un contrôle final avant livraison, mais comme un processus continu, intégré au cycle de sprint déjà posé en [partie 5.4](#) et prolongeant la logique d'amélioration continue de la [partie 6](#).

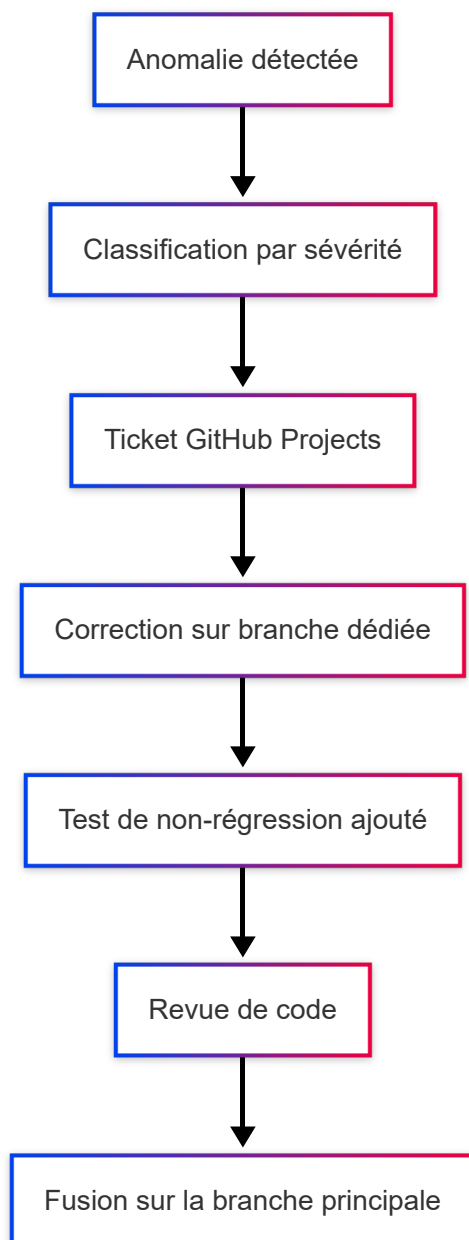
9.1 Détection et priorisation des anomalies

Une anomalie peut être détectée à trois niveaux distincts : automatiquement par les tests unitaires (Jest) et le linter (ESLint/Prettier) exécutés à chaque push via GitHub Actions (voir [partie 5.2](#)), manuellement lors des tests d'API sous Postman, ou remontée lors d'une démo de sprint. Quelle que soit son origine, chaque anomalie est classée selon sa sévérité avant d'être traitée :

Sévérité	Exemple	Délai de traitement visé
Bloquante	Impossible de planifier un itinéraire	Immédiat, avant toute autre tâche du sprint
Majeure	Un critère de classement (partie 7.2) n'est pas pris en compte	Dans le sprint en cours
Mineure	Décalage visuel sur un écran secondaire	Sprint suivant, si la priorité le permet

Cette classification évite de traiter chaque anomalie dans l'ordre d'arrivée : une anomalie bloquante sur le planificateur d'itinéraires (fonctionnalité cœur, [partie 4.4](#)) prend toujours le pas sur un défaut mineur.

9.2 Processus de correction



Chaque anomalie donne lieu à un ticket dans GitHub Projects, traité sur une branche dédiée plutôt que directement sur la branche principale, dans la continuité des pratiques déjà posées en [partie 5.4](#).

Une régression désigne un bogue déjà corrigé qui réapparaît plus tard, à cause d'une modification du code qui n'a a priori rien à voir avec lui. Exemple concret : un bogue empêche la prise en compte des perturbations GTFS-Realtime dans le classement des itinéraires ([partie 7.2](#)). Le correctif est accompagné d'un test qui vérifie précisément ce cas, et ce test reste ensuite en permanence dans la suite de tests, même une fois le bogue corrigé. Résultat : si une modification future du code venait à nouveau casser cette prise en compte, sans que personne ne s'en aperçoive directement, ce test échouerait automatiquement et alerterait avant la mise en production.

La fusion sur la branche principale reste enfin soumise à la même revue de code que n'importe quelle autre évolution.

9.3 Approche spécifique à la phase de préproduction

La phase de préproduction concentre une vigilance particulière, car c'est le dernier point de contrôle avant qu'un usager réel comme [Antoine](#) ou [Muriel](#) ne soit exposé à une régression. Trois pratiques structurent cette phase : un parcours de test manuel, où l'on utilise soi-même l'application comme le ferait un usager réel, sur un environnement proche de la production (mêmes flux GTFS/GBFS, mêmes contraintes réseau), un jeu de données réaliste plutôt que des données de test simplifiées, et un passage prioritaire sur les parcours critiques — inscription et profil de mobilité (F1), recherche d'itinéraire (F2), intégration transport (F3), et classement personnalisé par IA ([partie 7](#)).

Un gel des nouvelles fonctionnalités est observé dans les derniers jours précédant chaque mise en production : seules les corrections de bogues bloquants ou majeurs sont encore acceptées, afin de ne pas introduire un nouveau risque au moment même où l'on cherche à en réduire.

9.4 Cas concrets de bogues traités

Trois exemples réels illustrent le processus des parties 9.1 à 9.3, du symptôme à la mesure de fond.

Historique de trajets vide en production : variable d'environnement manquante. *Symptôme* : l'écran d'historique restait vide alors que des recherches avaient bien été faites. *Investigation* : reproduction locale, puis diagnostic sur le serveur : la clé de chiffrement au repos (`GEOLOCATION_ENCRYPTION_KEY` , [annexe E](#)) était **absente du fichier de configuration du serveur**. Comme ce fichier est maintenu à la main (hors dépôt, pour ne pas versionner de secrets), l'ajout d'une nouvelle variable au code ne se propage pas tout seul. Chaque écriture chiffrée échouait donc en silence, la fonction d'enregistrement de l'historique avalant l'erreur par conception (pour ne jamais faire échouer une recherche). *Correctif* : ajout de la variable côté serveur. *Mesure de fond* : une validation au démarrage qui **fait échouer le lancement** si un secret critique manque ou est mal dimensionné (plutôt qu'une dégradation invisible), et une étape de CI qui compare les variables du serveur à celles attendues et alerte en cas d'écart. Le même diagnostic a révélé que les clés de notifications push manquaient aussi, corrigé dans la foulée.

Rotation du refresh token : écart dossier / code. *Symptôme* : l'annexe C du dossier décrivait la rotation du refresh token comme un principe appliqué, alors que le code se contentait d'émettre un nouveau jeton sans invalider l'ancien (limite explicitement notée lors de l'audit OWASP). *Traitement* : traité comme une fonctionnalité à part entière : nouvelle table de suivi des jetons, détection de rejeu, tests dédiés (rotation nominale, rejeu, non-régression du flux de connexion). *Mesure de fond* : mise à jour du dossier pour que l'annexe C reflète l'état réel du code, et pas l'intention.

Repli "trajet à pied" partout : données de transport périmées. *Symptôme* : pendant une courte période, toutes les recherches ne renvoyaient qu'un itinéraire à pied. *Cause* : le flux GTFS chargé dans le moteur de routage a une fenêtre de validité glissante ; une fois dépassée, le moteur n'a plus aucun service à proposer et se rabat silencieusement sur la marche. *Mesure de fond* : un rafraîchissement automatique hebdomadaire du flux GTFS en production, pour que la fenêtre de validité ne se referme jamais sans intervention.

Point commun aux trois : le correctif ponctuel ne suffit pas : chaque cas se conclut par une mesure qui empêche la classe de problème de se reproduire silencieusement (validation au démarrage, vérification de CI, tâche planifiée), dans la logique de distinction entre anomalie ponctuelle et problème récurrent de la [partie 6.3](#).

10. Contraintes transverses (sécurité, RGPD, accessibilité, éco-conception, PWA, performance)

Ces contraintes s'appliquent à l'ensemble de la plateforme plutôt qu'à un module en particulier. Certaines ont déjà été évoquées ponctuellement dans le dossier (recueil des besoins en [partie 2.5](#), choix d'architecture en [partie 3](#)) ; cette partie les rassemble et détaille comment chacune se traduit concrètement dans la conception.

10.1 Sécurité des données

La sécurité s'appuie sur les standards OWASP déjà mentionnés en [partie 2.5](#), déclinés en pratiques concrètes : authentification par JWT avec refresh tokens **à rotation et détection de rejeu** et mots de passe hachés (bcrypt), déjà posée comme choix technique en [partie 3.10](#) (*fonctionnement détaillé en [annexe C](#) et [annexe D](#)*) ; validation systématique des données entrantes côté API pour se prémunir des injections ; communications chiffrées en HTTPS de bout en bout ; en-têtes de sécurité HTTP standards ; et une limitation du nombre de requêtes (rate limiting) sur les endpoints sensibles, notamment ceux liés à l'authentification. Ces points ont fait l'objet d'un audit OWASP dédié en cours de projet, qui a donné lieu à plusieurs correctifs (rate limiting, en-têtes, restriction explicite de l'algorithme de signature des jetons).

Les données de géolocalisation méritent une vigilance particulière : elles permettent de reconstituer des habitudes de déplacement (domicile, lieu de travail, horaires), ce qui en fait une catégorie de données particulièrement sensible si elle venait à fuiter. (*Cartographie détaillée des risques en [annexe E](#)*.)

10.2 RGPD et données de géolocalisation

Le respect du RGPD repose sur trois principes appliqués dès la conception (privacy by design) : la minimisation des données collectées (ne conserver que ce qui est strictement nécessaire au fonctionnement du planificateur), une durée de conservation limitée des trajets individuels, et un droit à l'effacement accessible directement depuis le profil de mobilité.

Pour les besoins statistiques évoqués en [partie 2.4](#) (orienter les décisions d'infrastructure de la métropole), les données utilisées seraient agrégées et anonymisées : la collectivité pourrait ainsi savoir qu'une ligne de bus est saturée aux heures de pointe sans que cela remonte à un usager identifiable. Ce traitement agrégé reste, à ce stade, un principe de conception plutôt qu'un tableau de bord livré : aucun canal dédié à la collectivité n'existe encore dans la version actuelle de la plateforme. L'hébergement en France ou en Europe, déjà justifié en [partie 3.9](#), complète cette conformité en évitant tout transfert de données hors UE.

10.3 Accessibilité (WCAG 2.1 AA)

L'accessibilité n'est pas traitée comme une contrainte réglementaire à part, mais comme un besoin direct d'un usager comme [Muriel](#) : contrastes de couleurs suffisants, navigation complète au clavier, taille de police ajustable, textes alternatifs sur les éléments visuels, et compatibilité avec les lecteurs d'écran.

Cette exigence dépasse le seul champ de l'interface : le profil de mobilité (F1, [partie 4.4](#)) permet déjà de signaler des contraintes concrètes prises en compte dans le calcul d'un trajet — accessibilité en fauteuil roulant, limitation de la distance de marche, limitation du nombre de correspondances. Le signalement d'obstacles ponctuels et changeants (un trottoir dégradé à un instant donné, par exemple) reste hors de portée réaliste pour cette version : il supposerait une source de données vivante sur l'état de la voirie, qu'aucun standard ouvert équivalent à GTFS/GBFS ne couvre aujourd'hui. (*Grille de conformité détaillée en [annexe F](#)*.)

10.4 Éco-conception

L'éco-conception s'appuie sur le référentiel RGEN (Référentiel Général d'Écoconception de Services Numériques), avec plusieurs leviers concrets : hébergement chez un fournisseur français aux engagements environnementaux documentés (OVHcloud, déjà argumenté en [partie 3.9](#)), allègement du frontend (chargement différé des composants peu utilisés via `React.lazy`, compression des assets au build), mise en cache des résultats d'itinéraires récents pour limiter les appels redondants au moteur de routage, et caches en mémoire côté backend pour les flux temps réel (GBFS, GTFS-Realtime) rafraîchis en tâche de fond plutôt qu'à chaque requête. Ces leviers restent des choix de conception : leur effet n'a pas été mesuré avec un outil dédié (type EcolIndex) sur ce périmètre.

Au-delà de ces leviers techniques, la plateforme sert par nature une cause écologique : chaque trajet reporté vers un mode de transport doux plutôt que la voiture individuelle constitue, à l'échelle de la métropole, un impact positif plus significatif que l'empreinte de l'application elle-même.

10.5 PWA et performance en mobilité

Le caractère PWA de la plateforme, déjà argumenté en [partie 3.4](#), répond directement à un usage en mobilité où la connectivité varie constamment : un service worker met en cache les derniers itinéraires consultés et une partie des données cartographiques, pour rester utilisable même en cas de perte de connexion temporaire.

L'application reste installable depuis le navigateur, sans passer par un store.

Le design responsive garantit une expérience cohérente quel que soit le support, ce qui compte directement pour un usager comme [Antoine](#), qui consulte la plateforme presque exclusivement depuis son téléphone. La performance sous connectivité variable est également recherchée activement, via le chargement progressif des données et un mode dégradé qui privilégie les informations essentielles (prochain trajet, alerte en cours) lorsque le réseau est faible.

11. Conclusion et perspectives

Il y a deux façons de regarder ce que devient UrbanFlow Mobility au terme de ce dossier.

- **Ce que voit l'utilisateur :**

Antoine ouvre la PWA depuis son téléphone, sans rien installer depuis un store. Il renseigne un trajet et obtient des itinéraires déjà classés selon ses critères et préférences ([partie 7.2](#)).

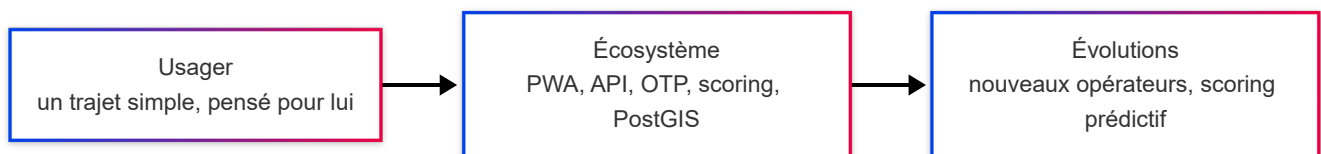
Muriel, elle, retrouve dans son profil de mobilité la possibilité de signaler qu'elle évite les correspondances — une contrainte prise en compte dès le calcul d'itinéraire. Elle n'est pas placée dans une case accessibilité ([partie 10.3](#)).

Un point de départ, une arrivée, un trajet qui tient compte de chaque utilisateur, ils ne sont pas placés derrière des profils pré-réglés. Ce que voit l'utilisateur reste volontairement simple.

- **Ce qui le rend possible :**

Cette simplicité repose sur un écosystème que ce dossier a détaillé pas à pas : une PWA React qui dialogue avec une API NestJS, elle-même orchestrant OpenTripPlanner et ses flux GTFS/GBFS ([partie 3.3](#)), un service de scoring qui pondère selon la météo et le profil utilisateur ([partie 7.3](#)), une base PostgreSQL/PostGIS conforme RGPD ([partie 10.2](#)).

Chaque brique peut évoluer sans faire bouger les autres — un découplage posé dès la [partie 3.5](#).



Concevoir une solution numérique, ce n'est pas seulement la faire fonctionner : c'est décider où placer la complexité pour qu'elle serve l'utilisateur plutôt qu'elle ne lui soit imposée.

C'est cette manière de raisonner, plus que la stack technique retenue, qui recoupe directement les compétences visées par le Titre 6 Concepteur Développeur de Solutions Digitales.

11.1 Perspectives et feuille de route post-MVP

Le périmètre livré est un socle : les fonctionnalités obligatoires (F1–F3) et une fonctionnalité au choix ([partie 7](#)). L'architecture a été pensée pour que les extensions suivantes se branchent sans réécriture (découplage [3.5](#), sources de données interchangeables [2.6](#) / [4.5](#)).

Évolution	Ce que ça demande	Dépendances
Nouveaux opérateurs / autres villes	Brancher un flux GTFS/GBFS conforme supplémentaire ; aucun code applicatif à modifier	—
Vélo/trottinette libre-service dans le calcul d'itinéraire	Aujourd'hui affichés sur la carte (disponibilité temps réel) mais pas encore intégrés au routage multimodal ; nécessite de configurer OpenTripPlanner avec le volet GBFS	Données GBFS déjà consommées
Réservation unifiée (vélos, trottinettes, places de covoiturage)	Nouveau module backend + tables dédiées ; s'appuie sur la même API et les comptes existants	API opérateurs (réservation), selon des conventions avec chaque opérateur
Calculateur d'empreinte carbone avec suivi personnel	Facteurs d'émission par mode (barème ADEME) appliqués aux segments d'un itinéraire, puis agrégation dans un tableau de bord citoyen	Historique de trajets (déjà persisté)
Covoiturage dynamique avec <i>matching</i>	Module de mise en relation + géomatching (PostGIS déjà en place) ; enjeu principal = masse critique d'usagers	—
Scoring prédictif	Passer de la somme pondérée à un modèle entraîné sur l'historique d'usage réel (partie 7.4)	Volume de données d'usage suffisant
Canal collectivité (tableau de bord agrégé)	Exposer des statistiques agrégées et anonymisées à la métropole (partie 10.2)	Cadre RGPD du traitement statistique à formaliser
Gamification / signalement collaboratif	Modules d'engagement, moindre priorité produit	—

Une même logique traverse cette liste : faire grandir l'écosystème sans jamais complexifier ce que l'utilisateur, lui, continue de voir.

11.2 Écarts, frictions et enseignements (post-mortem)

Ce dossier a d'abord été rédigé comme un document de conception amont (juillet), puis mis à jour en fin de projet : il fait aussi office de point de clôture. Ce bilan croise ce qui a bien fonctionné, les frictions rencontrées, les écarts entre la conception initiale et le produit livré, et ce qui serait fait autrement.

Ce qui a bien fonctionné

- Le **découplage frontend / backend** et le traitement des flux comme sources interchangeable ont tenu : aucune des fonctionnalités ajoutées en cours de route (scoring, suivi de perturbation, notifications) n'a demandé de revenir sur la structure en couches.
- Le **cycle itératif** avec revue de code systématique et tests de non-régression a permis de livrer en continu sans accumuler de dette bloquante : 151 *pull requests*, chacune revue, sur 7 semaines.
- Les **revues manuelles de bout en bout** en conditions réelles (mêmes données, mêmes contraintes réseau que la production) ont détecté des défauts qu'aucun test automatisé n'aurait vus (mauvais cadrage de carte, glyphes trop petits, messages d'erreur indifférenciés), et ont directement alimenté les sprints suivants.

Frictions et écarts

- **Configuration de production maintenue à la main.** Le fichier de configuration du serveur étant hors dépôt (pas de secrets versionnés), l'ajout d'une variable au code ne se propageait pas automatiquement : deux fonctionnalités (chiffrement de l'historique, notifications push) ont été cassées en silence en production avant d'être diagnostiquées ([partie 9.4](#)). Corrigé par une validation au démarrage et une vérification de CI, mais le coût de diagnostic aurait été évité par une gestion de configuration plus rigoureuse dès le départ.
- **Écarts dossier / code.** Certaines sections du dossier de juillet décrivaient des mécanismes comme acquis alors qu'ils étaient encore à l'état d'intention (rotation du refresh token, traitement statistique agrégé). Ces écarts ont été résorbés, soit en implémentant (rotation), soit en requalifiant explicitement en principe de conception (canal collectivité). Ils rappellent qu'un document de conception doit distinguer nettement ce qui est prévu de ce qui est fait.
- **Portée initiale surdimensionnée.** La demande client listait une dizaine de fonctionnalités ; le cadrage a resserré à F1–F3 + une fonctionnalité au choix. Ce resserrement a été le bon choix, mais il aurait pu être posé plus tôt et plus explicitement dans le dossier de juillet.
- **Estimation de la charge d'intégration.** Le déploiement d'OpenTripPlanner avec de vraies données GTFS/OSM et sa maintenance (fenêtre de validité glissante du flux) ont demandé plus de mise au point que prévu, d'où le rafraîchissement automatique hebdomadaire ajouté en cours de route.

Ce qui serait fait autrement

- Traiter la **configuration de production comme du code** dès le premier déploiement (fichier d'exemple versionné + vérification d'écart en CI en place *avant* la première mise en production, pas après un incident).
- **Figurer la portée dans le dossier** en distinguant systématiquement, pour chaque fonctionnalité, si elle est conçue, en cours, livrée ou hors périmètre.
- Mettre en place l'**audit d'accessibilité automatisé** dès le premier écran plutôt qu'à mi-parcours : plusieurs corrections auraient été prises en compte au fil de l'eau au lieu d'un rattrapage groupé.

12. Annexes

Les annexes qui suivent n'ont pas vocation à être lues pour comprendre les parties auxquelles elles se rattachent : chaque partie du dossier reste autonome, avec ses tableaux comparatifs et ses recommandations. Elles apportent un complément pour qui veut aller plus loin sur un point précis — origine d'un standard, fonctionnement détaillé d'un mécanisme de sécurité, ou grille de conformité — sans alourdir le corps du dossier.

- [Annexe A — Standards de données de transport, origine et adoption](#)
- [Annexe B — Moteurs de routage : fonctionnement et déploiements réels](#)
- [Annexe C — Authentification JWT et refresh tokens, fonctionnement](#)
- [Annexe D — Hachage des mots de passe avec bcrypt](#)
- [Annexe E — Cartographie des risques sur les données de géolocalisation](#)
- [Annexe F — Grille de conformité WCAG 2.1 AA](#)
- [Annexe G — Glossaire](#)

Annexe A — Standards de données de transport, origine et adoption

GTFS. Créé en 2005 à l'initiative de TriMet (l'autorité de transport de Portland, Oregon) en partenariat avec Google, sous le nom initial de "Google Transit Feed Specification". Le premier service Google Transit basé sur ce flux a été lancé le 7 décembre 2005 avec les seules données de TriMet ; cinq agences supplémentaires ont rejoint le dispositif dès 2006. En 2010, le standard a été renommé "General Transit Feed Specification" pour marquer son ouverture au-delà de Google, à mesure que son adoption s'accélérait. Aujourd'hui, plus de 10 000 opérateurs de transport dans le monde publient un flux GTFS, ce qui représente plus de 75 % des agences proposant des services de transport programmé.

GBFS. Créé en 2014 par Mitch Vars avec la collaboration d'opérateurs de vélo-partage, d'éditeurs d'applications et de fournisseurs technologiques. La version 1.0 a été portée par la NABSA (North American Bikeshare Association) et lancée en 2015. La gouvernance du standard a ensuite été confiée à MobilityData en 2019, avant un transfert complet de propriété en 2022. Le standard est aujourd'hui utilisé par plus de 730 systèmes de mobilité partagée dans plus de 40 pays.

GTFS-Realtime. Extension du GTFS statique au format Protocol Buffers (format binaire, plus compact qu'un CSV), publiée par Google quelques années après le GTFS initial pour couvrir les besoins de données temps réel (position des véhicules, retards, alertes).

Sources :

- [Predictors of Early Adoption of the General Transit Feed Specification — Findings Press](#)
- [Advancing Adoption of General Transit Feed Specification \(GTFS\) — Caltrans](#)
- [NABSA selects MobilityData as the new home of the GBFS specification — MobilityData](#)
- [GBFS — Get started](#)

Annexe B — Moteurs de routage : fonctionnement et déploiements réels

Fonctionnement d'OpenTripPlanner. OTP construit un graphe de routage à partir des flux GTFS (réseau de transport en commun) et des données OpenStreetMap (réseau piéton et cyclable), puis calcule les itinéraires multimodaux avec l'algorithme RAPTOR (*Round-based Public Transit Routing*), conçu pour gérer efficacement les correspondances entre lignes. Les mises à jour GTFS-Realtime peuvent être injectées pour ajuster les itinéraires proposés en fonction des perturbations en cours.

Déploiements existants. OpenTripPlanner est utilisé en production dans plusieurs contextes proches de celui d'UrbanFlow Mobility : en France, il alimente les services de plusieurs agglomérations dont Grenoble, Rennes et Alençon. À l'échelle nationale, la Norvège (Entur, depuis 2017, jusqu'à plus de 20 requêtes par seconde en heure de pointe) et la Finlande (Digitransit, porté par l'autorité de transport d'Helsinki) s'appuient sur OTP pour l'ensemble de leur réseau. Ces exemples confirment que l'outil tient la charge à l'échelle d'une métropole, ce qui conforte le choix fait en [partie 3.3](#).

Sources :

- [OpenTripPlanner Deployments Worldwide — documentation officielle](#)
- [OpenTripPlanner — dépôt officiel GitHub](#)

Annexe C — Authentification JWT et refresh tokens, fonctionnement

Principe du JWT. Un JSON Web Token est un jeton auto-porteur : signé par le serveur, il contient directement les informations nécessaires à son contrôle (identifiant utilisateur, date d'expiration) sans que le serveur ait besoin d'interroger une base de données à chaque requête pour vérifier une session. Cette approche sans état (*stateless*) est cohérente avec l'architecture découplée retenue en [partie 3.5](#), puisqu'elle facilite la scalabilité horizontale de l'API sans stockage de session partagé entre plusieurs instances du serveur.

Deux jetons, deux durées de vie. Le jeton d'accès (*access token*) est volontairement court (recommandation OWASP : entre 5 et 15 minutes selon la sensibilité des données), pour limiter la fenêtre d'exposition en cas de vol. Il est utilisé à chaque appel à l'API. Le jeton de rafraîchissement (*refresh token*), plus long à vivre, sert uniquement à obtenir un nouveau jeton d'accès une fois celui-ci expiré, sans redemander l'identifiant et le mot de passe à l'utilisateur.

Rotation du refresh token (implémentée). Pour limiter les conséquences d'un vol de jeton, l'OWASP recommande de faire tourner (*rotate*) le refresh token à chaque utilisation : un nouveau jeton est émis et l'ancien est immédiatement invalidé. Si un jeton déjà utilisé est présenté une seconde fois, cela signale une tentative de réutilisation frauduleuse (rejeu) et permet de révoquer l'ensemble de la session concernée.

Cette rotation a d'abord été documentée comme limite connue lors de l'audit OWASP, puis implémentée dans une itération suivante : chaque refresh token émis est tracé en base et rattaché à une *famille* (une session). À l'échange, le jeton présenté est marqué comme consommé et remplacé ; s'il est re-présenté au-delà d'une courte fenêtre de tolérance (qui absorbe les rafraîchissements quasi simultanés de plusieurs onglets), toute la famille est révoquée : la session entière est déconnectée. Les jetons expirés sont purgés automatiquement.

Cette combinaison (jeton d'accès très court, refresh token à rotation avec détection de rejeu, hébergement des données en France, [partie 3.9](#)) est particulièrement pertinente pour UrbanFlow Mobility, dont les données de géolocalisation figurent parmi les plus sensibles de la plateforme (voir [partie 10.1](#)) : réduire la durée de validité d'un jeton d'accès réduit d'autant la fenêtre pendant laquelle un jeton intercepté pourrait être exploité pour accéder à l'historique de déplacement d'un usager.

Sources :

- [JSON Web Token Cheat Sheet — OWASP](#)
- [OAuth2 Cheat Sheet — OWASP](#)

Annexe D — Hachage des mots de passe avec bcrypt

Hacher n'est pas chiffrer. Un mot de passe n'est jamais stocké en clair, ni même chiffré (ce qui suppose une clé pour le déchiffrer) : il est haché, c'est-à-dire transformé par une fonction à sens unique impossible à inverser. Lors

de la connexion, le mot de passe saisi est haché à nouveau et le résultat comparé au hachage stocké, sans que le mot de passe d'origine n'ait besoin d'être conservé nulle part.

Le rôle du sel. Chaque mot de passe est haché avec un sel (une valeur aléatoire unique, générée et gérée automatiquement par bcrypt) ajouté avant le hachage. Sans sel, deux usagers ayant le même mot de passe obtiendraient le même hachage, ce qui permettrait de le retrouver via des tables précalculées (*rainbow tables*). Avec un sel propre à chaque compte, deux hachages restent différents même pour un mot de passe identique.

Le facteur de travail. Contrairement à une fonction de hachage classique (rapide par nature), bcrypt est volontairement lent, et ce ralentissement est réglable via un facteur de travail (*work factor*) : plus il est élevé, plus le calcul est coûteux, ce qui ralentit d'autant les tentatives de force brute en cas de fuite de la base de données. La recommandation OWASP actuelle est un facteur de travail d'au moins 12, contre 10 par défaut dans la plupart des implémentations.

Une limite à connaître. L'OWASP recommande aujourd'hui Argon2 comme premier choix pour une nouvelle application, bcrypt restant cependant une option reconnue et largement supportée, en particulier dans l'écosystème Node.js/NestJS retenu en [partie 3.7](#). Ce choix reste donc un compromis assumé entre robustesse et simplicité d'intégration, plutôt que l'option la plus récente disponible.

Sources :

- [Password Storage Cheat Sheet — OWASP](#)

Annexe E — Cartographie des risques sur les données de géolocalisation

Cette cartographie détaille les principaux risques identifiés autour des données de géolocalisation, la catégorie de données la plus sensible de la plateforme (voir [partie 10.1](#)), et la solution retenue pour répondre à chacun.

Risque	Scénario	Solution retenue
Interception réseau	Un trajet est capté en clair entre la PWA et l'API, par exemple sur un réseau Wi-Fi public non sécurisé	Chiffrement HTTPS de bout en bout sur l'ensemble des échanges (partie 10.1)
Fuite de la base de données	Un accès non autorisé à la base PostgreSQL/PostGIS expose l'historique de trajets de l'ensemble des usagers	Chiffrement applicatif au repos des coordonnées et libellés d'adresse (AES-256-GCM, clé dédiée hors base, vecteur d'initialisation aléatoire à chaque écriture) : historique de trajets, adresses domicile/travail du profil, trajet suivi. Accès à la base restreint aux seuls services qui en ont besoin
Ré-identification par recoupement	Même anonymisées, des données de trajets agrégées à un niveau trop fin (ex. un trajet domicile-travail unique) peuvent permettre de ré-identifier une personne	Agrégation à un niveau géographique et temporel suffisamment large avant tout usage statistique (partie 10.2)
Compromission de l'appareil de l'utilisateur	Le cache hors-ligne de la PWA (partie 10.5) contient les derniers trajets consultés ; un vol ou un accès physique au téléphone expose ces données	Durée de vie limitée du cache local, contenu minimal (pas d'historique complet, seulement les derniers trajets utiles au mode dégradé)
Exposition via un service tiers	Un appel mal conçu vers un service externe (météo, GTFS-Realtime) transmettrait une position précise plutôt qu'une zone générale	Les appels aux services tiers (partie 7.3) sont limités aux données strictement nécessaires à leur fonction, sans transmission de l'identité de l'utilisateur
Accès interne excessif	Un accès trop large aux données de géolocalisation par l'équipe technique, au-delà du strict besoin de maintenance	Séparation des rôles (partie 5.3) et traçabilité des accès aux données sensibles

Cette cartographie reste volontairement centrée sur les scénarios les plus directement liés à l'architecture retenue dans ce dossier, plutôt que sur une analyse de risque exhaustive au sens d'une méthode formelle (EBIOS RM, STRIDE), hors de portée réaliste pour un projet individuel à ce stade de conception.

Annexe F — Grille de conformité WCAG 2.1 AA

Le WCAG 2.1 (Web Content Accessibility Guidelines) est le référentiel de référence en matière d'accessibilité numérique, structuré en 50 critères de succès répartis sur trois niveaux (A, AA, AAA). Le niveau AA, évoqué en [partie 10.3](#), est le niveau généralement exigé dans les obligations légales (notamment l'obligation d'accessibilité numérique en France). Le tableau ci-dessous ne reprend pas les 50 critères mais sélectionne ceux qui ont une traduction concrète directe dans la conception d'UrbanFlow Mobility, pensée pour un usager comme [Muriel](#).

Critère WCAG 2.1	Exigence	Application dans UrbanFlow Mobility
1.1.1 Contenu non textuel	Toute information non textuelle (icône, pictogramme, image) doit avoir une alternative textuelle	Les pictogrammes de mode de transport (bus, tram, vélo) et les icônes d'alerte sur la carte disposent d'un texte alternatif lu par les lecteurs d'écran
1.4.3 Contraste (minimum)	Un rapport de contraste d'au moins 4,5:1 entre le texte et son arrière-plan	La charte graphique de l'application est vérifiée avec un outil de contrôle de contraste avant intégration, en particulier sur les badges de statut (perturbation, retard)
1.4.4 Redimensionnement du texte	Le texte doit rester lisible et fonctionnel jusqu'à 200% de zoom, sans perte de contenu	L'interface est construite en unités relatives (rem/em) plutôt qu'en pixels fixes, pour permettre l'agrandissement sans rupture de mise en page
1.4.10 Reflow	Le contenu doit se réorganiser sur un seul axe (pas de défilement horizontal) même à fort zoom ou sur petit écran	Cohérent avec le design responsive déjà posé en partie 10.5 : la mise en page s'adapte à la largeur disponible sans défilement horizontal
1.4.11 Contraste des éléments non textuels	Les éléments d'interface (boutons, icônes, bordures de champs) doivent aussi respecter un contraste minimal	Les boutons d'action (lancer une recherche, valider un itinéraire) et les champs de formulaire conservent un contraste suffisant même hors état de focus
2.1.1 Clavier	Toute fonctionnalité doit être utilisable au clavier seul, sans dépendre de la souris ou du tactile	Le parcours complet (recherche d'itinéraire, gestion du profil de mobilité F1) reste accessible par tabulation, ce qui profite aussi à un usage avec clavier externe sur tablette
2.4.6 En-têtes et étiquettes	Les titres de section et les étiquettes de champs doivent décrire clairement leur contenu ou leur fonction	Les champs du profil de mobilité (préférences, obstacles à éviter) portent des libellés explicites plutôt que de simples icônes seules
2.4.7 Visibilité du focus	L'élément actuellement sélectionné au clavier doit être visuellement identifiable	Un contour de focus visible est conservé sur tous les éléments interactifs, sans être supprimé pour des raisons esthétiques
2.5.3 Label dans le nom	Le nom accessible d'un élément doit inclure le texte visible qui le désigne	Important pour la compatibilité avec les commandes vocales, pertinent pour un usage mains libres en mobilité comme celui d' Antoine

Critère WCAG 2.1	Exigence	Application dans UrbanFlow Mobility
3.3.1 / 3.3.2 Identification des erreurs et instructions	Les erreurs de saisie doivent être signalées clairement, et les champs doivent être accompagnés d'instructions	Le formulaire d'inscription et la recherche d'itinéraire indiquent explicitement le champ en erreur et la nature du problème, plutôt qu'un message générique
4.1.2 Nom, rôle, valeur	Les composants d'interface personnalisés (carte interactive, sélecteurs) doivent exposer correctement leur état aux technologies d'assistance	Les composants non standards (carte, sélecteur de préférences du profil de mobilité) sont construits avec les attributs ARIA nécessaires plutôt qu'en HTML non sémantique

Cette sélection n'a pas vocation à remplacer un audit d'accessibilité complet, mais à intégrer les critères les plus structurants dès la conception plutôt que de les traiter en correction a posteriori. En complément, une suite d'audit automatisée (Playwright + axe-core) rejoue ces vérifications sur les écrans clés à chaque push ([partie 5.2](#)) et fait échouer la CI en cas de régression d'accessibilité.

Sources :

- [W3C — Web Content Accessibility Guidelines \(WCAG\) 2.1](#)
- [W3C — WCAG 2.1 Quick Reference \(liste filtrable des critères de succès\)](#)

Annexe G — Glossaire

Terme	Définition
GTFS (<i>General Transit Feed Specification</i>)	Format ouvert décrivant l'offre de transport en commun <i>théorique</i> (lignes, arrêts, horaires planifiés). Standard de fait, alimenté par la plupart des autorités de transport.
GTFS-Realtime (GTFS-RT)	Extension temps réel de GTFS : perturbations, retards, positions de véhicules. Utilisée ici pour détecter un incident sur une ligne suivie.
GBFS (<i>General Bikeshare Feed Specification</i>)	Équivalent de GTFS pour les mobilités en libre-service : position et disponibilité en temps réel des vélos et trottinettes en station.
OpenTripPlanner (OTP)	Moteur open source de calcul d'itinéraires multimodaux, à partir des flux GTFS/GBFS et des données OpenStreetMap (annexe B).
RAPTOR (<i>Round-bAsed Public Transit Optimized Router</i>)	Algorithme de calcul d'itinéraires en transport en commun par tours successifs, optimisé pour les correspondances. Utilisé par OpenTripPlanner.
Dijkstra / A*	Algorithmes classiques de plus court chemin dans un graphe, utilisés pour le routage sur le réseau piéton/cyclable.
Somme pondérée multicritère (<i>Weighted Sum Model</i>)	Méthode d'aide à la décision : chaque critère reçoit un poids explicite, le score global est leur combinaison linéaire. Base du service de scoring (partie 7.3).
PWA (<i>Progressive Web App</i>)	Application web installable, dotée d'un <i>service worker</i> pour le fonctionnement hors ligne partiel, sans passer par un magasin d'applications.
Service worker	Script exécuté par le navigateur en arrière-plan de la page : gère le cache hors ligne et la réception des notifications push.
JWT (<i>JSON Web Token</i>)	Jeton d'authentification auto-porteur et signé, vérifiable sans consulter la base (annexe C).
VAPID (<i>Voluntary Application Server Identification</i>)	Paire de clés identifiant le serveur applicatif auprès du service de push du navigateur, requise pour l'envoi de notifications Web Push.
PostGIS	Extension géospatiale de PostgreSQL : types géométriques, index et requêtes spatiales natives.
AES-256-GCM	Algorithme de chiffrement symétrique <i>authentifié</i> (détecte toute altération du texte chiffré). Utilisé pour le chiffrement au repos des données de géolocalisation (annexe E).
MaaS (<i>Mobility as a Service</i>)	Modèle agrégeant l'ensemble des offres de mobilité d'un territoire dans une application unique (partie 2.1).
RGESN	Référentiel Général d'Écoconception de Services Numériques (référentiel public français).